



Специалист Kafka с глубоким пониманием системы

Модуль 1 - Основы Kafka изнутри

Урок 1 - Архитектура Kafka простыми словами — брокеры, кластер, топики

Шаг 02:

Вопрос 1

Что такое Kafka брокер?

Подсказка: Представьте, что брокер — это почтовое отделение, куда приходят сообщения и откуда они распределяются.

Ответ:

- **Сервер, который хранит и обрабатывает сообщения в Kafka**
- Программа для отправки писем по email
- Устройство для майнинга криптовалюты
- Средство для редактирования кода

Шаг 03:

Вопрос 2

Что образуют несколько брокеров Kafka, работающих вместе?

Подсказка: В одиночку брокер слаб, но вместе они составляют мощную систему.

Ответ:

- **Кластер**
- Файл конфигурации
- Базу данных
- Портативный сервер

🔗 **Шаг 04:**

Вопрос 3

Что такое топик в Kafka?

Подсказка: Топик похож на папку, в которую складываются сообщения определённого типа.

Ответ:

- **Логическое хранилище сообщений определённой категории**
- Обычный текстовый файл на диске
- Имя компьютера в сети
- Команда для запуска сервера

🔗 **Шаг 05:**

Вопрос 4

Сопоставьте элементы Kafka с их правильным описанием:

Ответ:

- Брокер
- Кластер
- Топик
- Продюсер

Подсказка: Подумайте: продюсер отправляет сообщения, брокер их хранит, кластер — это несколько брокеров, а топик — это категория сообщений.

- **Брокер** — Сервер, который принимает и хранит сообщения
- **Кластер** — Набор брокеров, работающих вместе
- **Топик** — Категория сообщений
- **Продюсер** — Программа, которая отправляет сообщения

🔗 **Шаг 06:**

Вопрос 5

Кто в Kafka читает сообщения из топика?

Подсказка: Если продюсер пишет — значит есть тот, кто читает.

Ответ:

- **Консюмер**
- Парсер
- Брокер
- Скрипт Bash

🔗 **Шаг 07:**

Вопрос 6

Сопоставьте участников системы Kafka с их ролью:

Ответ:

- Продюсер
- Консюмер
- Брокер
- Топик

Подсказка: Продюсер пишет → брокер хранит → топик структурирует → консюмер читает.

- **Продюсер** — Отправляет данные
- **Консюмер** — Читает данные
- **Брокер** — Принимает и сохраняет данные
- **Топик** — Группирует сообщения по категориям

 Шаг 08:

Вопрос 7

Почему в Kafka топики делятся на партиции?

Подсказка: Подумайте о том, как распределить нагрузку и ускорить обработку больших потоков данных.

Ответ:

- **Чтобы можно было обрабатывать сообщения параллельно**
- Чтобы сообщения выглядели красивее
- Чтобы уменьшить количество топиков
- Чтобы сообщения автоматически удалялись

 Шаг 09:

Вопрос 8

Расположите процесс доставки сообщения в Kafka в правильном порядке:

1. Сообщение хранится в партиции топика
2. Продюсер отправляет сообщение
3. Брокер принимает сообщение
4. Консюмер читает сообщение

Подсказка: Сначала сообщение создаётся, потом попадает на сервер, сохраняется, и только после этого читается.

Ответ:

- Продюсер отправляет сообщение
- Брокер принимает сообщение
- Сообщение хранится в партиции топика
- Консюмер читает сообщение

 Шаг 10:

Вопрос 9

Что такое репликация в Kafka?

Подсказка: Представьте, что у вас есть копии важных файлов на разных флешках, чтобы не потерять их.

Ответ:

- **Создание копий партиций на разных брокерах**

- Отправка сообщений дважды для надёжности
- Дублирование топиков вручную
- Создание новых кластеров при ошибках

🔗 Шаг 11:

Вопрос 10

Расположите шаги работы кластера Kafka с репликацией:

1. Лидер партиции принимает сообщение
2. Фолловеры получают копию сообщения
3. Сообщение подтверждается как сохранённое
4. Консюмер читает сообщение

Подсказка: Вспомните: сначала лидер, потом фолловеры, потом подтверждение, потом чтение.

Ответ:

- Лидер партиции принимает сообщение
- Фолловеры получают копию сообщения
- Сообщение подтверждается как сохранённое
- Консюмер читает сообщение

📖 Урок 2 - Как Kafka хранит сообщения — журнал (лог) вместо таблицы

🔗 Шаг 02:

Вопрос 1

Как Kafka хранит сообщения внутри себя?

Подсказка: Представьте дневник: записи добавляются в конец и не меняются.

Ответ:

- **В журнале (логе), где новые записи добавляются в конец**
- В таблицах как в SQL-базе
- В случайных папках на диске
- В виде картинок

🔗 Шаг 03:

Вопрос 2

Почему Kafka выбрала журнал (лог), а не таблицу для хранения?

Подсказка: Подумайте о скорости добавления и простоте последовательного чтения.

Ответ:

- **Потому что записи быстро добавляются в конец и легко читаются последовательно**
- Чтобы хранить картинки
- Чтобы хранить только одну запись
- Чтобы экономить место за счёт удаления данных

🔗 Шаг 04:

Вопрос 3

Сопоставьте понятия Kafka с их объяснением:

Ответ:

- Лог (журнал)
- Запись
- Смещение (offset)
- Партиция

Подсказка: Лог — это последовательность. Запись — одно сообщение. Смещение — номер записи. Партиция — кусочек топика.

- **Лог (журнал)** — Последовательное хранилище сообщений
- **Запись** — Одно сообщение внутри лога
- **Смещение (offset)** — Порядковый номер сообщения
- **Партиция** — Раздел топика, который содержит часть записей

 Шаг 05:

Вопрос 4

Как консюмер понимает, какое сообщение он уже прочитал?

Подсказка: Каждый элемент в логе имеет уникальный порядковый номер.

Ответ:

- **Он запоминает смещение (offset) последнего прочитанного сообщения**
- Он смотрит на дату сохранения файла
- Он хранит список всех сообщений вручную
- Он спрашивает брокера наугад

 Шаг 06:

Вопрос 5

Что произойдёт со старыми сообщениями в Kafka?

Подсказка: У Kafka есть политика хранения: либо по времени, либо по размеру.

Ответ:

- **Они могут быть удалены по политике хранения (например, через 7 дней)**
- Они сохраняются навсегда, пока не выключат сервер
- Они превращаются в новые сообщения
- Они сразу теряются после чтения

 Шаг 07:

Вопрос 6

Сопоставьте политики хранения сообщений с их определением:

Ответ:

- retention.ms
- retention.bytes
- log.compaction

Подсказка: Одна политика — по времени, другая — по размеру, третья — оставляет последние значения ключей.

- **retention.ms** — Удаление старых сообщений через заданное время
- **retention.bytes** — Удаление при превышении объёма
- **log.compaction** — Хранение только последнего сообщения для каждого ключа

Шаг 08:

Вопрос 7

Расположите шаги записи сообщения в Kafka:

1. Продюсер отправляет сообщение
2. Брокер записывает сообщение в лог
3. Сообщение получает смещение (offset)
4. Консьюмер читает сообщение из лога

Подсказка: Сначала отправка, потом запись, затем присвоение номера, и только потом чтение.

Ответ:

- Продюсер отправляет сообщение
- Брокер записывает сообщение в лог
- Сообщение получает смещение (offset)
- Консьюмер читает сообщение из лога

Шаг 09:

Вопрос 8

Почему консьюмеры могут читать одно и то же сообщение независимо друг от друга?

Подсказка: У каждого консьюмера есть собственный указатель на смещение.

Ответ:

- **Потому что у каждого консьюмера есть свой offset**
- Потому что брокер копирует сообщения для каждого
- Потому что сообщения читаются только один раз
- Потому что Kafka автоматически дублирует все топики

Шаг 10:

Вопрос 9

Что даёт использование лога вместо таблицы в плане аналитики?

Подсказка: Подумайте: можно заново проиграть все события от начала.

Ответ:

- **Возможность воспроизвести поток событий заново для анализа**
- Сокращение всех сообщений в одну таблицу
- Удаление старых данных без следа
- Автоматическое превращение данных в графики

Шаг 11:

Вопрос 10

Расположите шаги при повторном воспроизведении лога для аналитики:

1. Консьюмер начинает читать с `offset = 0`
2. Сообщения читаются последовательно
3. Аналитическая программа обрабатывает каждое сообщение
4. Делаются выводы и строятся отчёты

Подсказка: Сначала читаем с самого начала, затем идём по порядку, обрабатываем, и в конце получаем результат.

Ответ:

- Консьюмер начинает читать с `offset = 0`
- Сообщения читаются последовательно
- Аналитическая программа обрабатывает каждое сообщение
- Делаются выводы и строятся отчёты

Урок 3 - Репликация и отказоустойчивость — пример с дежурными в классе

Шаг 02:

Вопрос 1

В классе несколько дежурных. Если один заболел, другой может заменить его. Какой принцип Kafka это иллюстрирует?

Подсказка: Подумай про надёжность системы при сбое.

Ответ:

- **Репликация**
- Балансировка нагрузки
- Шардирование
- Кэширование

Шаг 03:

Вопрос 2

Если у каждой парты свой дежурный, а у каждого дежурного есть помощник — это похоже на?

Подсказка: Вспомни про разделы (партиции) и их копии.

Ответ:

- **Партиции с репликами**
- Только один брокер
- Асинхронные вызовы
- Индексация сообщений

Шаг 04:

Вопрос 3

Сопоставь роли дежурных с терминами Kafka:

Подсказка: У дежурного-лидера есть главная ответственность, у помощника — запасная.

Ответ:

- **Главный дежурный** — Лидер партиции
- **Помощник дежурного** — Фолловер партиции
- **Ведение порядка в классе** — Обслуживание потока данных
- **Замена при болезни** — Отказоустойчивость

Шаг 05:

Вопрос 4

Если главный дежурный ушёл домой, как система класса (и Kafka) должна себя повести?

Подсказка: Вспомни про выбор нового лидера.

Ответ:

- **Назначить помощника главным**
- Оставить класс без дежурного
- Подождать, пока вернётся главный
- Отменить дежурства

🔗 **Шаг 06:**

Вопрос 5

Расположи шаги выбора нового лидера после сбоя:

1. Фиксируется сбой текущего лидера
2. Выбирается новый лидер из реплик
3. Оповещаются остальные брокеры
4. Клиенты перенаправляются на нового лидера

Подсказка: Подумай: сначала замечаем проблему, потом выбираем замену, потом уведомляем всех, и только затем клиенты работают с новым лидером.

Ответ:

- Фиксируется сбой текущего лидера
- Выбирается новый лидер из реплик
- Оповещаются остальные брокеры
- Клиенты перенаправляются на нового лидера

🔗 **Шаг 07:**

Вопрос 6

Почему важно, чтобы реплики были актуальными и синхронизированными?

Подсказка: Вспомни, что будет, если помощник не в курсе последних событий.

Ответ:

- **Чтобы при сбое лидера не потерять данные**
- Чтобы сообщения быстрее удалялись
- Чтобы ученикам было веселее
- Чтобы уменьшить количество партиций

🔗 **Шаг 08:**

Вопрос 7

Сопоставь термины с их объяснением:

Подсказка: Вспомни, кто главный, кто подстраховывает и что означает отказоустойчивость.

Ответ:

- **Лидер** — Основной брокер, принимающий записи
- **Фолловер** — Реплика, копирующая данные лидера
- **ISR (in-sync replicas)** — Реплики, которые успевают синхронно обновляться
- **Failover** — Автоматическое переключение при сбое

🔗 **Шаг 09:**

Вопрос 8

Что произойдёт, если фолловер сильно отстанет от лидера?

Подсказка: Вспомни про список ISR — там остаются только те, кто успевает копировать.

Ответ:

- **Его уберут из ISR**
- Он станет новым лидером
- Он удалит все данные
- Он будет дежурным навсегда

Шаг 10:

Вопрос 9

Расположи шаги репликации сообщения:

1. Продюсер отправляет сообщение лидеру
2. Лидер записывает сообщение в лог
3. Фолловеры копируют данные у лидера
4. Клиент получает подтверждение

Подсказка: Сначала пишем в лидера, потом дублируем, а уж после подтверждаем.

Ответ:

- Продюсер отправляет сообщение лидеру
- Лидер записывает сообщение в лог
- Фолловеры копируют данные у лидера
- Клиент получает подтверждение

Шаг 11:

Вопрос 10

Если в классе только один дежурный без помощников, чем это грозит?

Подсказка: Подумай про риск потери данных при отсутствии реплик.

Ответ:

- **При сбое потеряются все данные**
- Система станет быстрее
- Фолловеры появятся автоматически
- Ничего страшного не произойдёт

Урок 4 - Мини-тест 1 «Что делает брокер»

Шаг 01:

Вопрос 1

Ученик приносит тетради и раскладывает их по парте. На кого похож брокер в Kafka?

Подсказка: Он не пишет сам, а хранит и передаёт.

Ответ:

- **На хранителя сообщений**
- На создателя сообщений
- На учителя
- На проверяющего домашку

Шаг 02:

Вопрос 2

Кто отправляет сообщения в брокера?

Подсказка: Это программы-«писатели».

Ответ:

- **Продюсеры**
- Консьюмеры
- Фолловеры
- Учителя

Шаг 03:

Вопрос 3

Что хранит брокер внутри себя?

Подсказка: Подумай про журнал Kafka.

Ответ:

- **Сообщения в виде логов**
- Только таблицы
- Только ключи без данных
- Ничего не хранит

Шаг 04:

Вопрос 4

Сопоставь понятия и роли:

Подсказка: Вспомни: кто пишет, кто хранит, кто читает.

Ответ:

- **Продюсер** — Отправляет сообщение
- **Брокер** — Хранит сообщение
- **Консьюмер** — Читает сообщение
- **Партиция** — Раздел внутри топика

Шаг 05:

Вопрос 5

Если брокеров несколько, зачем они нужны вместе?

Подсказка: Думай про масштаб и надёжность.

Ответ:

- **Чтобы распределять нагрузку и повышать отказоустойчивость**
- Чтобы мешать друг другу
- Чтобы хранить только один топик
- Чтобы делать систему медленнее

Шаг 06:

Вопрос 6

Расположи действия при обработке сообщения брокером:

1. Продюсер отправляет сообщение
2. Брокер принимает и записывает его в лог
3. Консьюмер читает сообщение
4. Брокер удаляет старые сообщения по настройкам

Подсказка: Сначала запись, потом чтение, а удаление — самое последнее.

Ответ:

- Продюсер отправляет сообщение
- Брокер принимает и записывает его в лог
- Консьюмер читает сообщение
- Брокер удаляет старые сообщения по настройкам

Шаг 07:

Вопрос 7

Почему брокер не удаляет сразу сообщение после чтения?

Подсказка: Подумай, что консьюмеров может быть несколько.

Ответ:

- **Чтобы другие консьюмеры тоже могли его прочитать**
- Потому что забыл
- Чтобы увеличить размер лога
- Чтобы продюсер перечитал сообщение

Шаг 08:

Вопрос 8

Сопоставь функции брокера:

Подсказка: У брокера несколько обязанностей: хранение, передача и управление.

Ответ:

- **Хранение** — Запись сообщений в лог
- **Передача** — Отправка сообщений консьюмерам
- **Масштабирование** — Работа в кластере
- **Отказоустойчивость** — Поддержка реплик

Шаг 09:

Вопрос 9

Если брокер «упал», что делает кластер Kafka?

Подсказка: Вспомни про репликацию и лидерство.

Ответ:

- **Назначает нового лидера из реплик**
- Останавливает работу навсегда
- Удаляет все сообщения
- Ждёт пока вернётся старый брокер

Шаг 10:

Вопрос 10

Расположи шаги при добавлении нового брокера в кластер:

1. Запуск нового брокера
2. Регистрация в кластере
3. Распределение партиций и реплик
4. Обслуживание клиентов

Подсказка: Сначала брокер стартует, потом встраивается в систему, получает нагрузку и уже после обслуживает клиентов.

Ответ:

- Запуск нового брокера
- Регистрация в кластере
- Распределение партиций и реплик
- Обслуживание клиентов

Урок 5 - Мини-тест 2 «Что делает брокер»

Шаг 01:

Вопрос 1

Кто в Kafka принимает сообщения от продюсеров и хранит их?

Подсказка: Это главная роль брокера.

Ответ:

- **Брокер**
- Топик
- Клиент
- Зоопаркер

Шаг 02:

Вопрос 2

Можно ли в кластере Kafka иметь несколько брокеров?

Подсказка: Чем больше брокеров, тем выше масштабируемость.

Ответ:

- **Да, это обычная практика**
- Нет, всегда только один

Шаг 03:

Вопрос 3

Сопоставь термин и его объяснение:

Подсказка: Брокер работает с топиками и партициями.

Ответ:

- **Брокер** — Сервер, хранящий данные Kafka
- **Топик** — Логическая категория сообщений
- **Партиция** — Физическое деление топика внутри брокера
- **Клиент** — Тот, кто пишет или читает данные

Шаг 04:

Вопрос 4

Что хранит брокер для каждого топика?

Подсказка: Данные делятся на независимые части для параллельной работы.

Ответ:

- **Партиции**
- Кодеки
- Метаданные клиента
- Графики нагрузки

Шаг 05:

Вопрос 5

Расположи шаги доставки сообщения через брокер:

1. Продюсер отправляет сообщение
2. Брокер сохраняет его в партиции
3. Консьюмер запрашивает данные
4. Брокер выдаёт сообщение консьюмеру

Подсказка: Сначала сообщение попадает в брокер, потом хранится, а уже после читается.

Ответ:

- Продюсер отправляет сообщение
- Брокер сохраняет его в партиции
- Консьюмер запрашивает данные
- Брокер выдаёт сообщение консьюмеру

Шаг 06:

Вопрос 6

Если один брокер перегружен, что обычно делает Kafka?

Подсказка: Вспомни про распределение партиций между брокерами.

Ответ:

- **Размещает партиции на других брокерах**
- Удаляет часть данных
- Отключает консьюмеров
- Выключает весь кластер

Шаг 07:

Вопрос 7

Сопоставь действие брокера с его назначением:

Подсказка: Брокер — посредник между продюсером и консьюмером.

Ответ:

- **Принятие сообщений** — Работа с продюсерами
- **Сохранение сообщений** — Запись данных в партиции
- **Выдача сообщений** — Обслуживание консьюмеров
- **Репликация** — Поддержка отказоустойчивости

Шаг 08:

Вопрос 8

Как брокер понимает, к какой партиции записать сообщение?

Подсказка: Обычно используется ключ или хэш.

Ответ:

- **Использует ключ сообщения для выбора партиции**
- Записывает всё в первую партицию
- Выбирает случайно
- Сохраняет в файл без партиций

Шаг 09:

Вопрос 9

Расположи шаги работы брокера при репликации:

1. Лидер брокер получает сообщение
2. Лидер сохраняет его в лог
3. Фолловеры копируют сообщение у лидера
4. Лидер подтверждает приём продюсеру

Подсказка: Сначала лидер принимает, потом фиксирует, затем дублируют фолловеры, и только потом идёт подтверждение.

Ответ:

- Лидер брокер получает сообщение
- Лидер сохраняет его в лог
- Фолловеры копируют сообщение у лидера
- Лидер подтверждает приём продюсеру

Шаг 10:

Вопрос 10

Что произойдёт, если брокер, где хранится лидер партиции, выйдет из строя?

Подсказка: Вспомни про фолловеров и выбор нового лидера.

Ответ:

- **Один из фолловеров станет новым лидером**
- Все данные будут потеряны
- Продюсер заменит брокера
- Кластер остановится навсегда

Модуль 2 - Лог и сегменты

Урок 1 - Лог — последовательность сообщений

Шаг 02:

Вопрос 1

Что в Kafka называют логом?

Подсказка: Это не журнал в школе, а структура данных для хранения сообщений.

Ответ:

- **Последовательность сообщений в порядке записи**
- Список ошибок
- Таблица в базе данных
- Случайный набор данных

🔮 **Шаг 03:**

Вопрос 2

Как сообщения в логе упорядочены?

Подсказка: Вспомни школьную очередь в столовую.

Ответ:

- **По времени записи одно за другим**
- В случайном порядке
- Сначала новые, потом старые
- По алфавиту

🔮 **Шаг 04:**

Вопрос 3

Сопоставь термин и его описание:

Подсказка: В логе есть уникальные номера, порядок и хранение.

Ответ:

- **Лог** — Последовательность сообщений
- **Оффсет** — Номер сообщения в логе
- **Порядок** — Сообщения идут друг за другом
- **Сегмент** — Часть большого лога

🔮 **Шаг 05:**

Вопрос 4

Почему Kafka делит лог на сегменты?

Подсказка: Представь длинную тетрадь, которую неудобно хранить цельной.

Ответ:

- **Чтобы упростить хранение и удаление старых данных**
- Чтобы перепутать сообщения
- Чтобы сделать систему медленнее
- Чтобы было меньше сообщений

🔮 **Шаг 06:**

Вопрос 5

Расположи шаги при записи сообщения в лог:

1. Продюсер отправляет сообщение
2. Брокер получает сообщение
3. Сообщение добавляется в конец сегмента лога
4. Сообщение получает оффсет

Подсказка: Сначала отправка, потом приём, потом запись и присвоение номера.

Ответ:

- Продюсер отправляет сообщение
- Брокер получает сообщение
- Сообщение добавляется в конец сегмента лога
- Сообщение получает оффсет

🔗 Шаг 07:

Вопрос 6

Что происходит, когда сегмент лога переполняется?

Подсказка: Как в тетради — заводим новую, а старую откладываем.

Ответ:

- **Открывается новый сегмент для записи**
- Удаляются все сообщения
- Брокер останавливается
- Система теряет данные

🔗 Шаг 08:

Вопрос 7

Сопоставь сегмент и его характеристики:

Подсказка: У сегмента есть начало, конец и свой размер.

Ответ:

- **Начальный оффсет** — Номер первого сообщения в сегменте
- **Конечный оффсет** — Номер последнего сообщения в сегменте
- **Файл данных** — Хранит сами сообщения
- **Файл индекса** — Помогает быстро найти сообщение

🔗 Шаг 09:

Вопрос 8

Почему оффсет в Kafka не сбрасывается после чтения?

Подсказка: Подумай про нескольких читателей одного и того же лога.

Ответ:

- **Чтобы разные консьюмеры могли читать одно и то же сообщение независимо**
- Потому что брокер забывает сбросить
- Чтобы увеличить нагрузку
- Чтобы продюсер перечитал сообщение

🔗 Шаг 10:

Вопрос 9

Расположи действия при удалении старых сегментов:

1. Проверка возраста или размера сегмента
2. Пометка сегмента как устаревшего
3. Удаление файлов сегмента
4. Освобождение места на диске

Подсказка: Сначала проверка, потом пометка, потом удаление и освобождение памяти.

Ответ:

- Проверка возраста или размера сегмента
- Пометка сегмента как устаревшего
- Удаление файлов сегмента
- Освобождение места на диске

 **Шаг 11:**

Вопрос 10

Зачем Kafka использует сегменты вместо единого огромного файла?

Подсказка: Подумай про удобство управления, поиска и удаления.

Ответ:

- **Чтобы упростить поиск, удаление и управление данными**
- Чтобы тратить больше памяти
- Чтобы сообщения путались
- Чтобы брокер работал медленнее

Урок 2 - Сегменты логов — части журнала для удобства хранения

 **Шаг 02:**

Вопрос 1

Почему Kafka разбивает лог на сегменты?

Подсказка: Подумайте, как удобнее хранить и удалять данные: в одной большой книге или в нескольких томах?

Ответ:

- **Чтобы упрощать хранение и удаление старых данных**
- Чтобы продюсер всегда писал быстрее
- Чтобы сообщения не терялись
- Чтобы делать сообщения короче

 **Шаг 03:**

Вопрос 2

Когда сегмент лога считается готовым к удалению?

Подсказка: Вспомните про политику хранения сообщений (retention).

Ответ:

- **Когда превышено время или размер сегмента**
- Когда продюсер перестал писать в топик
- Когда потребитель прочитал хотя бы одно сообщение
- Когда брокер перезапускается

 **Шаг 04:**

Вопрос 3

Что хранится внутри сегмента лога Kafka?

Подсказка: Подумайте о тетради с записями и указателями страниц.

Ответ:

- **Сообщения и индекс для поиска**
- Только сообщения
- Только индексы
- Метаданные топика без сообщений

🔗 Шаг 05:

Вопрос 4 Расположите шаги работы сегмента лога по порядку:

1. Запись нового сообщения
2. Обновление индекс-файла
3. Проверка retention
4. Удаление старого сегмента

🔗 Подсказка:

Сначала записываем данные, обновляем индекс, проверяем условие retention и удаляем старые сегменты.

Ответ:

- Запись нового сообщения
- Обновление индекс-файла
- Проверка retention
- Удаление старого сегмента

🔗 Шаг 06:

Вопрос 5

Как Kafka быстро находит сообщение в сегменте?

Подсказка: Используйте аналогию с оглавлением книги.

Ответ:

- **Через индекс-файл offset → позиция**
- Последовательно перебирая все сообщения
- По имени продюсера
- По timestamp сегмента, игнорируя offset

🔗 Шаг 07:

Вопрос 6

Сопоставьте элементы сегмента с функцией:

Ответ:

- Offset
- Message
- Index-файл

Подсказка: Ассоциация с номерами страниц, содержимым и оглавлением книги.

- **Offset** — Позиция сообщения в сегменте
- **Message** — Содержимое, которое отправил продюсер
- **Index-файл** — Файл для быстрого поиска по offset

🔮 Шаг 08:

Вопрос 7

Почему сегменты помогают масштабировать Kafka?

Подсказка: Подумайте о больших объемах сообщений и быстром доступе.

Ответ:

- **Старые сегменты удаляются, новые создаются параллельно**
- Сегменты заменяют брокеры
- Сегменты ускоряют продюсера, игнорируя потребителя
- Сегменты уменьшают объем сообщений

🔮 Шаг 09:

Вопрос 8 Расположите действия сегмента при удалении старых данных:

1. Проверка условий retention
2. Закрытие сегмента
3. Удаление файлов сегмента
4. Освобождение ресурсов брокера

🔮 **Подсказка:**

Сначала проверяем, потом закрываем, удаляем файлы и освобождаем ресурсы.

Ответ:

- Проверка условий retention
- Закрытие сегмента
- Удаление файлов сегмента
- Освобождение ресурсов брокера

🔮 Шаг 10:

Вопрос 9

Сопоставьте тип файлов сегмента с содержимым:

Ответ:

- .log
- .index
- .timeindex

Подсказка: Подумайте о назначении каждого файла.

- **.log** — Основные сообщения
- **.index** — Соответствие offset → позиция
- **.timeindex** — Соответствие timestamp → позиции сообщения

🔮 Шаг 11:

Вопрос 10

Что обеспечивает отказоустойчивость сегментов?

Подсказка: Вспомните про репликацию и резервные копии.

Ответ:

- **Репликация сегментов на другие брокеры**

- Хранение сегментов только в одном файле
- Использование одного продюсера
- Удаление сегментов чаще обычного

Урок 3 - Практика визуализируем лог на доске

Шаг 02:

Вопрос 1

Что отображает визуализация лога на доске?

Подсказка: Подумайте, как можно наглядно показать последовательность сообщений.

Ответ:

- **Последовательность сообщений в топике**
- Состояние брокеров
- Список всех потребителей
- Время работы продюсера

Шаг 03:

Вопрос 2

Почему полезно визуализировать сегменты лога на доске?

Подсказка: Представьте доску с блоками, где каждая часть показывает часть журнала.

Ответ:

- **Чтобы увидеть, какие сегменты старые и могут быть удалены**
- Чтобы изменить содержимое сообщений
- Чтобы ускорить продюсер
- Чтобы выключить брокеры

Шаг 04:

Вопрос 3

Какая информация обычно указывается в визуализации сегмента?

Подсказка: Подумайте о блоках с сообщениями и их идентификаторах.

Ответ:

- **Offset, размер и состояние сегмента**
- IP-адрес продюсера
- Количество тем в топике
- Время ожидания потребителя

Шаг 05:

Вопрос 4 Расположите действия при визуализации сегмента:

1. Выбрать сегмент
2. Отобразить сообщения
3. Отметить offset
4. Проверить состояние retention

Подсказка:

Сначала выбираем сегмент, показываем его содержимое, отмечаем offset и проверяем, что можно удалить.

Ответ:

- Выбрать сегмент
- Отобразить сообщения
- Отметить offset
- Проверить состояние retention

🔗 Шаг 06:

Вопрос 5

Как визуализация помогает при обучении Kafka?

Подсказка: Подумайте о наглядности последовательности и структуры сегментов.

Ответ:

- **Помогает понять, как сообщения организованы в сегменты**
- Помогает редактировать сообщения в логе
- Автоматически удаляет старые сегменты
- Ускоряет продюсер

🔗 Шаг 07:

Вопрос 6

Сопоставьте элементы визуализации с их значением:

Ответ:

- Блок сегмента
- Цифра на блоке
- Цвет блока

Подсказка: Используйте аналогию с книгой: страницы, номера и пометки состояния.

- **Блок сегмента** — Группа сообщений
- **Цифра на блоке** — Offset первого сообщения
- **Цвет блока** — Состояние сегмента (активный/старый)

🔗 Шаг 08:

Вопрос 7

Почему полезно менять цвет сегментов в визуализации?

Подсказка: Подумайте о наглядном различии старых и новых сегментов.

Ответ:

- **Чтобы быстро увидеть старые сегменты, готовые к удалению**
- Чтобы выделить сегменты с ошибками в сообщениях
- Чтобы ускорить запись продюсера
- Чтобы сделать доску красивее

🔗 Шаг 09:

Вопрос 8 Расположите шаги при визуализации нескольких сегментов:

1. Выбор топика
2. Выбор сегментов
3. Отображение сообщений сегментов

4. Проверка их состояния

Подсказка:

Сначала выбираем топик, потом сегменты, показываем сообщения и проверяем состояние.

Ответ:

- Выбор топика
- Выбор сегментов
- Отображение сообщений сегментов
- Проверка их состояния

Шаг 10:

Вопрос 9

Сопоставьте визуальные элементы с их функцией:

Ответ:

- Толстая линия
- Тонкая линия
- Разделитель сегмента

Подсказка: Аналогия с линиями и границами в книге.

- **Толстая линия** — Показывает конец старого сегмента
- **Тонкая линия** — Разделяет сообщения внутри сегмента
- **Разделитель сегмента** — Отделяет один сегмент от другого

Шаг 11:

Вопрос 10

Что дает полное визуальное представление лога на доске?

Подсказка: Подумайте о контроле, обучении и анализе состояния сегментов.

Ответ:

- **Помогает анализировать последовательность и состояние сегментов для обучения и контроля**
- Автоматически исправляет ошибки сообщений
- Ускоряет запись продюсера без участия брокера
- Удаляет старые сегменты без retention

Урок 4 - Мини-тест «Как сообщения делятся на сегменты»

Шаг 02:

Вопрос 1

Что такое сегмент в Kafka?

Подсказка: Подумайте о частях лога для удобного хранения сообщений.

Ответ:

- **Часть лога, содержащая последовательность сообщений**
- Брокер Kafka
- Потребитель сообщений
- Топик Kafka

Шаг 03:

Вопрос 2

Почему сообщения делятся на сегменты?

Подсказка: Подумайте о том, как лог хранится и управляется в Kafka.

Ответ:

- **Чтобы эффективно управлять хранением и удалением старых сообщений**
- Чтобы ускорить продюсер
- Чтобы менять содержимое сообщений
- Чтобы создавать топики автоматически

Шаг 04:

Вопрос 3

Что обозначает offset в сегменте?

Подсказка: Подумайте о номере сообщения внутри сегмента.

Ответ:

- **Уникальный номер сообщения внутри топика**
- Размер сегмента
- Цвет сегмента на доске
- Время жизни сообщения

Шаг 05:

Вопрос 4 Расположите действия при создании нового сегмента:

1. Лог достигает максимального размера сегмента
2. Создание нового сегмента
3. Продюсер начинает писать в новый сегмент
4. Старый сегмент готов к удалению при необходимости

 **Подсказка:**

Сначала сегмент достигает лимита, потом создается новый, туда пишутся новые сообщения, а старый остается для retention.

Ответ:

- Лог достигает максимального размера сегмента
- Создание нового сегмента
- Продюсер начинает писать в новый сегмент
- Старый сегмент готов к удалению при необходимости

Шаг 06:

Вопрос 5

Что делает retention для сегментов?

Подсказка: Подумайте о том, как Kafka управляет временем хранения сообщений.

Ответ:

- **Удаляет старые сегменты после достижения срока хранения или размера**
- Создает новые топики
- Меняет offset сообщений

- Сохраняет все сегменты навсегда

🔗 Шаг 07:

Вопрос 6

Сопоставьте элементы сегмента с их значением:

Ответ:

- Offset
- Сегмент
- Retention

Подсказка: Подумайте о номере, блоке сообщений и сроке хранения.

- **Offset** — Номер сообщения в топике
- **Сегмент** — Группа сообщений в лог-файле
- **Retention** — Срок хранения сегмента

🔗 Шаг 08:

Вопрос 7

Почему Kafka не хранит все сообщения в одном сегменте?

Подсказка: Подумайте о масштабируемости и управлении логами.

Ответ:

- **Чтобы управлять размером файлов и временем хранения более эффективно**
- Чтобы все продюсеры писали в один файл
- Чтобы ускорить удаление топиков
- Чтобы изменить порядок сообщений

🔗 Шаг 09:

Вопрос 8 Расположите шаги обработки сегментов для удаления старых сообщений:

1. Проверка времени создания сегмента
2. Сравнение с параметром retention
3. Маркировка сегмента как устаревшего
4. Удаление устаревшего сегмента

🔗 **Подсказка:**

Сначала проверяем дату, сравниваем с retention, отмечаем устаревший и удаляем.

Ответ:

- Проверка времени создания сегмента
- Сравнение с параметром retention
- Маркировка сегмента как устаревшего
- Удаление устаревшего сегмента

🔗 Шаг 10:

Вопрос 9

Сопоставьте типы сегментов с их назначением:

Ответ:

- Активный сегмент

- Старый сегмент
- Удаляемый сегмент

Подсказка: Подумайте о том, куда пишет продюсер и какие сегменты уже завершены.

- **Активный сегмент** — Текущий сегмент, куда пишутся новые сообщения
- **Старый сегмент** — Сегмент, уже не принимающий новые сообщения
- **Удаляемый сегмент** — Сегмент, срок хранения которого истек

Шаг 11:

Вопрос 10

Что важно помнить при делении сообщений на сегменты в Kafka?

Подсказка: Подумайте о последовательности, retention и эффективности хранения.

Ответ:

- **Сегменты должны быть управляемыми по размеру, времени хранения и сохранению последовательности сообщений**
- Все сообщения должны храниться в одном сегменте
- Segmentation влияет только на цвет визуализации
- Retention применяется только к топикам, а не к сегментам

Модуль 3 - Партиции и ключи сообщений

Урок 1 - Партиции — отдельные «коридоры» сообщений

Шаг 02:

Вопрос 1

Если топик — это улица, то что такое партиция?

Подсказка: Подумай про отдельные «коридоры» для сообщений.

Ответ:

- **Отдельный «коридор» внутри топика**
- Сам топик
- Сообщение
- Кластер брокеров

Шаг 03:

Вопрос 2

Почему топик делят на партиции?

Подсказка: Подумай про скорость и масштабирование.

Ответ:

- **Чтобы параллельно обрабатывать сообщения**
- Чтобы сделать лог длиннее
- Чтобы хранить меньше данных
- Чтобы уменьшить количество брокеров

Шаг 04:

Вопрос 3

Что делает ключ сообщения в Kafka?

Подсказка: Подумай про определение партиции для сообщения.

Ответ:

- **Определяет, в какую партицию попадёт сообщение**
- Определяет, кто прочитает сообщение
- Шифрует сообщение
- Удаляет сообщение из партиции

🔮 **Шаг 05:**

Вопрос 4

Сопоставьте термины и их описание:

Подсказка: Подумай про структуру топика и роли ключей.

Ответ:

- **Партиция** — Отдельный канал сообщений внутри топика
- **Топик** — Общее место для сообщений одной темы
- **Ключ сообщения** — Определяет партицию
- **Офсет** — Позиция сообщения внутри партиции

🔮 **Шаг 06:**

Вопрос 5

Если два сообщения имеют одинаковый ключ, что произойдет?

Подсказка: Думай про порядок в партиции.

Ответ:

- **Они попадут в одну партицию и сохраняют порядок**
- Они попадут в разные партиции
- Первое сообщение удалится
- Ключ не влияет на партицию

🔮 **Шаг 07:**

Вопрос 6

Расположите шаги доставки сообщения с ключом в правильном порядке:

1. Продюсер отправляет сообщение с ключом
2. Брокер определяет партицию по ключу
3. Сообщение записывается в лог партиции
4. Консьюмер читает сообщение

Подсказка: Сперва отправка, потом выбор партиции, запись и чтение.

Ответ:

- Продюсер отправляет сообщение с ключом
- Брокер определяет партицию по ключу
- Сообщение записывается в лог партиции
- Консьюмер читает сообщение

🔮 **Шаг 08:**

Вопрос 7

Что произойдет, если партиций меньше, чем ключей?

Подсказка: Подумай про распределение по хэш-функции.

Ответ:

- **Некоторые ключи будут попадать в одну партицию**
- Каждый ключ получит отдельную партицию
- Сообщения не будут доставляться
- Брокер создаст новые партиции автоматически

🔮 Шаг 09:

Вопрос 8

Сопоставьте термины с их функциями в партициях:

Подсказка: Обратите внимание, кто что хранит и контролирует.

Ответ:

- **Партиция** — Последовательная запись сообщений
- **Ключ** — Определяет размещение сообщения
- **Офсет** — Позиция сообщения для чтения
- **Консьюмер** — Считывает сообщения из партиции

🔮 Шаг 10:

Вопрос 9

Зачем важно сохранять порядок сообщений внутри партиции?

Подсказка: Подумай про логи и последовательность событий.

Ответ:

- **Чтобы консьюмеры видели события в правильном порядке**
- Чтобы продюсер мог переписать сообщение
- Чтобы партиция занимала меньше места
- Чтобы брокеры быстрее удаляли сообщения

🔮 Шаг 11:

Вопрос 10

Расположите шаги, как Kafka распределяет сообщения по партициям:

1. Получение сообщения с ключом
2. Применение хэш-функции к ключу
3. Определение партиции
4. Запись сообщения в лог партиции

Подсказка: Сперва сообщение приходит, потом хэширует ключ, выбирает партицию и пишет в лог.

Ответ:

- Получение сообщения с ключом
- Применение хэш-функции к ключу
- Определение партиции
- Запись сообщения в лог партиции

Урок 2 - Ключи — способ направлять сообщения в нужную партицию

Шаг 02:

Вопрос 1

Что делает ключ сообщения в Kafka?

Подсказка: Подумай, как система решает, в какую партицию отправить сообщение.

Ответ:

- **Определяет, в какую партицию попадёт сообщение**
- Определяет, кто прочитает сообщение
- Шифрует сообщение
- Удаляет сообщение из партиции

Шаг 03:

Вопрос 2

Если у двух сообщений одинаковый ключ, что произойдет?

Подсказка: Думай про порядок сообщений внутри партиции.

Ответ:

- **Они попадут в одну партицию и сохраняют порядок**
- Они попадут в разные партиции
- Первое сообщение удалится
- Ключ не влияет на партицию

Шаг 04:

Вопрос 3

Что произойдет, если у сообщения нет ключа?

Подсказка: Подумай про способ распределения по партициям.

Ответ:

- **Kafka выберет партицию случайным образом или по round-robin**
- Сообщение не будет записано
- Брокер создаст новый ключ автоматически
- Сообщение попадет в первую партицию всегда

Шаг 05:

Вопрос 4

Сопоставьте термины с их функцией:

Подсказка: Подумай, кто что делает при отправке сообщений с ключом.

Ответ:

- **Ключ сообщения** — Определяет партицию для сообщения
- **Партиция** — Хранит последовательность сообщений
- **Офсет** — Позиция сообщения внутри партиции
- **Продюсер** — Отправляет сообщения с ключом

Шаг 06:

Вопрос 5

Зачем используют ключи в топиках с несколькими партициями?

Подсказка: Подумай про группировку сообщений и обработку по порядку.

Ответ:

- **Чтобы сообщения с одинаковым ключом попадали в одну партицию**
- Чтобы уменьшить количество партиций
- Чтобы ускорить продюсер
- Чтобы брокер мог удалить ключи

Шаг 07:

Вопрос 6

Расположите шаги доставки сообщения с ключом:

1. Продюсер отправляет сообщение с ключом
2. Брокер применяет хэш-функцию к ключу
3. Определяется партиция
4. Сообщение записывается в лог партиции
5. Консьюмер читает сообщение

Подсказка: Сначала отправка, потом хэширование и запись, затем чтение.

Ответ:

- Продюсер отправляет сообщение с ключом
- Брокер применяет хэш-функцию к ключу
- Определяется партиция
- Сообщение записывается в лог партиции
- Консьюмер читает сообщение

Шаг 08:

Вопрос 7

Если ключи сообщений равномерно распределены, что произойдет с партициями?

Подсказка: Подумай про баланс нагрузки.

Ответ:

- **Нагрузка будет распределена примерно равномерно между партициями**
- Все сообщения попадут в одну партицию
- Ключи будут игнорироваться
- Сообщения будут теряться

Шаг 09:

Вопрос 8

Сопоставьте ключи и их влияние:

Подсказка: Ключи решают, где и в каком порядке будут храниться сообщения.

Ответ:

- **Одинаковые ключи** — Попадают в одну партицию и сохраняют порядок
- **Разные ключи** — Могут попасть в разные партиции
- **Отсутствие ключа** — Партиция выбирается случайно
- **Равномерные ключи** — Нагрузка распределяется равномерно

Шаг 10:

Вопрос 9

Зачем важно правильно подбирать ключи в продакшн-топиках?

Подсказка: Подумай про баланс нагрузки и сохранение порядка сообщений.

Ответ:

- **Чтобы избежать перегрузки отдельных партиций и сохранить порядок**
- Чтобы ускорить брокер
- Чтобы продюсер не использовал ключи
- Чтобы ключи можно было игнорировать

Шаг 11:

Вопрос 10

Расположите шаги обработки сообщения с ключом, начиная с продюсера:

1. Продюсер формирует сообщение с ключом
2. Брокер применяет хэш-функцию к ключу
3. Определение целевой партиции
4. Запись сообщения в лог партиции
5. Консьюмер считывает сообщение

Подсказка: Сначала формирование, затем хэширование, выбор партиции, запись и чтение.

Ответ:

- Продюсер формирует сообщение с ключом
- Брокер применяет хэш-функцию к ключу
- Определение целевой партиции
- Запись сообщения в лог партиции
- Консьюмер считывает сообщение

Урок 3 - Практическое задание оценки или новости по партициям

Шаг 02:

Вопрос 1

Если вы отправляете оценки студентов с ключом «студент_id», что произойдет?

Подсказка: Подумайте, как ключ влияет на партицию.

Ответ:

- **Все оценки одного студента попадут в одну партицию**
- Оценки попадут в случайные партиции
- Оценки будут объединены в одно сообщение
- Сообщения будут удалены

Шаг 03:

Вопрос 2

Вы отправляете новости по партициям без ключа. Как Kafka распределит сообщения?

Подсказка: Подумайте про распределение нагрузки.

Ответ:

- **Сообщения попадут в партиции по round-robin или случайно**
- Все сообщения попадут в первую партицию
- Все сообщения попадут в одну партицию по алфавиту
- Kafka создаст новый ключ автоматически

Шаг 04:

Вопрос 3

Что произойдет, если ключ «студент_id» одинаковый для нескольких оценок?

Подсказка: Подумайте про порядок сообщений внутри партиции.

Ответ:

- **Все сообщения попадут в одну партицию и сохранят порядок**
- Сообщения попадут в разные партиции
- Первое сообщение удалится
- Порядок сообщений не гарантирован

Шаг 05:

Вопрос 4

Сопоставьте элементы системы с их функцией:

Подсказка: Подумайте, кто что делает при распределении сообщений по партициям.

Ответ:

- **Ключ сообщения** — Определяет партицию для сообщения
- **Партиция** — Хранит последовательность сообщений
- **Офсет** — Позиция сообщения внутри партиции
- **Продюсер** — Отправляет сообщения с ключом

Шаг 06:

Вопрос 5

Почему важно использовать ключи при отправке оценок студентов?

Подсказка: Подумайте про порядок и согласованность данных.

Ответ:

- **Чтобы все оценки одного студента попадали в одну партицию и порядок сохранялся**
- Чтобы уменьшить количество партиций
- Чтобы продюсер мог игнорировать сообщения
- Чтобы брокер мог удалить ключи

Шаг 07:

Вопрос 6

Расположите шаги обработки сообщения с оценкой студента:

1. Продюсер отправляет сообщение с ключом «студент_id»
2. Брокер применяет хэш-функцию к ключу
3. Определяется партиция
4. Сообщение записывается в лог партиции

5. Консьюмер считывает сообщение

Подсказка: Сначала отправка, потом хэширование, определение партиции, запись и чтение.

Ответ:

- Продюсер отправляет сообщение с ключом «студент_id»
- Брокер применяет хэш-функцию к ключу
- Определяется партиция
- Сообщение записывается в лог партиции
- Консьюмер считывает сообщение

Шаг 08:

Вопрос 7

Что произойдет, если ключи студентов распределены неравномерно?

Подсказка: Подумайте про нагрузку на партиции.

Ответ:

- **Некоторые партиции будут перегружены, другие пустыми**
- Все партиции будут использоваться равномерно
- Ключи будут проигнорированы
- Сообщения будут потеряны

Шаг 09:

Вопрос 8

Сопоставьте ключи и их влияние на партиции:

Подсказка: Ключи определяют порядок и распределение.

Ответ:

- **Одинаковые ключи** — Попадают в одну партицию и сохраняют порядок
- **Разные ключи** — Могут попасть в разные партиции
- **Отсутствие ключа** — Партиция выбирается случайно
- **Равномерные ключи** — Нагрузка распределяется равномерно

Шаг 10:

Вопрос 9

Почему важно проверять распределение ключей перед запуском продакшн-топика с оценками?

Подсказка: Подумайте про баланс нагрузки и порядок сообщений.

Ответ:

- **Чтобы избежать перегрузки отдельных партиций и сохранить порядок сообщений**
- Чтобы продюсер не использовал ключи
- Чтобы уменьшить количество сообщений
- Чтобы ключи можно было игнорировать

Урок 4 - Мини-тест «Что определяет порядок сообщений»

Шаг 02:

Вопрос 1

Что определяет, в каком порядке сообщения будут сохраняться в Kafka?

Подсказка: Думайте про партиции и ключи сообщений.

Ответ:

- **Порядок определяется внутри партиции, ключи влияют на выбор партиции**
- Сообщения всегда сохраняются в порядке отправки вне зависимости от ключей
- Порядок определяется случайно
- Ключи никак не влияют на порядок сообщений

🔮 Шаг 03:

Вопрос 2

Если два сообщения имеют одинаковый ключ, что гарантировано?

Подсказка: Подумайте про последовательность сообщений внутри партиции.

Ответ:

- **Они попадут в одну партицию и сохраняют порядок**
- Могут попасть в разные партиции
- Первое сообщение удалится
- Порядок не гарантируется

🔮 Шаг 04:

Вопрос 3

Что происходит с сообщениями без ключа?

Подсказка: Думайте про распределение по партициям и порядок сообщений.

Ответ:

- **Сообщения распределяются по партициям случайным образом или round-robin**
- Все сообщения попадут в первую партицию
- Сообщения будут удалены
- Kafka создаст ключ автоматически

🔮 Шаг 05:

Вопрос 4

Сопоставьте элементы Kafka с их ролью в определении порядка сообщений:

Подсказка: Думайте, кто влияет на последовательность внутри партиции.

Ответ:

- **Ключ сообщения** — Определяет, в какую партицию попадет сообщение
- **Партиция** — Гарантирует порядок сообщений внутри себя
- **Офсет** — Позиция сообщения в партиции
- **Продюсер** — Отправляет сообщения с указанным ключом

🔮 Шаг 06:

Вопрос 5

Почему одинаковые ключи полезны для обработки потоков данных по категориям?

Подсказка: Подумайте про согласованность и порядок обработки сообщений.

Ответ:

- **Чтобы все сообщения одной категории попадали в одну партицию и обрабатывались последовательно**

- Чтобы уменьшить количество партиций
- Чтобы брокер мог удалять сообщения
- Чтобы ключи игнорировались системой

Шаг 07:

Вопрос 6

Расположите шаги обработки сообщения с ключом:

1. Продюсер формирует сообщение с ключом
2. Брокер применяет хэш-функцию к ключу
3. Определяется партиция
4. Сообщение записывается в лог партиции
5. Консьюмер считывает сообщение

Подсказка: Сначала формирование сообщения, потом хэширование, выбор партиции, запись и чтение.

Ответ:

- Продюсер формирует сообщение с ключом
- Брокер применяет хэш-функцию к ключу
- Определяется партиция
- Сообщение записывается в лог партиции
- Консьюмер считывает сообщение

Шаг 08:

Вопрос 7

Что произойдет, если ключи распределены неравномерно?

Подсказка: Подумайте про нагрузку на партиции.

Ответ:

- **Некоторые партиции будут перегружены, а другие почти пустыми**
- Все партиции будут использоваться равномерно
- Сообщения будут потеряны
- Ключи будут игнорированы

Шаг 09:

Вопрос 8

Сопоставьте ключи и их влияние на порядок сообщений:

Подсказка: Ключи определяют, где и в каком порядке будут храниться сообщения.

Ответ:

- **Одинаковые ключи** — Попадают в одну партицию и сохраняют порядок
- **Разные ключи** — Могут попасть в разные партиции
- **Отсутствие ключа** — Партиция выбирается случайно
- **Равномерные ключи** — Нагрузка распределяется равномерно

Шаг 10:

Вопрос 9

Почему важно проверять распределение ключей перед запуском продакшн-топика?

Подсказка: Подумайте про баланс нагрузки и сохранение порядка сообщений.

Ответ:

- **Чтобы избежать перегрузки отдельных партиций и сохранить порядок сообщений**
- Чтобы ключи можно было игнорировать
- Чтобы продюсер не использовал ключи
- Чтобы уменьшить количество сообщений

 **Шаг 11:**

Вопрос 10

Расположите шаги доставки сообщений с ключами для гарантии порядка:

1. Продюсер формирует сообщение с ключом
2. Брокер применяет хэш-функцию
3. Определяется целевая партиция
4. Запись сообщения в лог партиции
5. Консьюмер считывает сообщение

Подсказка: Сначала формирование, потом хэширование, выбор партиции, запись и чтение.

Ответ:

- Продюсер формирует сообщение с ключом
- Брокер применяет хэш-функцию
- Определяется целевая партиция
- Запись сообщения в лог партиции
- Консьюмер считывает сообщение

Модуль 4 - Репликация и лидерство

Урок 1 - Лидер и реплики — кто отвечает за что

 **Шаг 02:**

Вопрос 1

Кто в Kafka отвечает за запись сообщений в топик?

Подсказка: Подумайте, какая реплика принимает записи от продюсеров.

Ответ:

- **Лидер партиции**
- Фолловер
- Продюсер
- Потребитель

 **Шаг 03:**

Вопрос 2

Что делает фолловер в репликации Kafka?

Подсказка: Подумайте о том, как данные синхронизируются между репликами.

Ответ:

- **Копирует данные с лидера для обеспечения отказоустойчивости**
- Принимает сообщения от продюсеров

- Удаляет старые сегменты
- Отвечает на запросы потребителей первым

🔗 Шаг 04:

Вопрос 3

Почему лидер важен для согласованности данных в Kafka?

Подсказка: Подумайте о том, как Kafka гарантирует, что все реплики получают одинаковые сообщения.

Ответ:

- **Потому что все записи проходят через лидера и фолловеры копируют их**
- Потому что лидер хранит только часть сообщений
- Потому что лидер отвечает за удаление старых сегментов
- Потому что лидер выполняет только чтение сообщений

🔗 Шаг 05:

Вопрос 4 Расположите шаги, которые происходят при записи сообщения в реплицированную партицию:

1. Продюсер отправляет сообщение лидеру
2. Лидер записывает сообщение в сегмент
3. Фолловеры копируют сообщение с лидера
4. Лидер подтверждает запись продюсеру

🔗 Подсказка:

Сначала продюсер пишет лидеру, лидер сохраняет, фолловеры копируют, и только после этого приходит подтверждение.

Ответ:

- Продюсер отправляет сообщение лидеру
- Лидер записывает сообщение в сегмент
- Фолловеры копируют сообщение с лидера
- Лидер подтверждает запись продюсеру

🔗 Шаг 06:

Вопрос 5

Что произойдет, если лидер партиции недоступен?

Подсказка: Подумайте о механизме выбора нового лидера.

Ответ:

- **Фолловер, имеющий актуальные данные, становится новым лидером**
- Все записи теряются
- Продюсер записывает данные напрямую в старые фолловеры
- Ничего не произойдет, лидер автоматически восстановится

🔗 Шаг 07:

Вопрос 6

Сопоставьте роли в репликации с их функцией:

Ответ:

- Лидер
- Фолловер
- Продюсер

Подсказка: Подумайте о том, кто пишет, кто копирует, кто отправляет сообщения.

- **Лидер** — Принимает записи и распространяет их фолловерам
- **Фолловер** — Копирует данные с лидера для синхронизации
- **Продюсер** — Отправляет сообщения в топик через лидера

Шаг 08:

Вопрос 7

Почему важно, чтобы все фолловеры синхронизировались с лидером?

Подсказка: Подумайте о надежности и отказоустойчивости системы.

Ответ:

- **Чтобы новый лидер мог быть выбран без потери данных**
- Чтобы увеличить скорость продюсера
- Чтобы изменить offset сообщений
- Чтобы удалить старые сегменты быстрее

Шаг 09:

Вопрос 8 Расположите события при переключении лидера:

1. Текущий лидер недоступен
2. Выбор нового лидера среди актуальных фолловеров
3. Продюсеры и потребители перенастраиваются на нового лидера
4. Система продолжает работу без потери сообщений

 **Подсказка:**

Сначала лидер недоступен, затем выбирают нового, все перенастраиваются, и работа продолжается.

Ответ:

- Текущий лидер недоступен
- Выбор нового лидера среди актуальных фолловеров
- Продюсеры и потребители перенастраиваются на нового лидера
- Система продолжает работу без потери сообщений

Шаг 10:

Вопрос 9

Сопоставьте термин с его значением в репликации Kafka:

Ответ:

- ISR
- Лидер
- Фолловер

Подсказка: Подумайте о списке синхронизированных реплик и функциях лидера/фолловеров.

- **ISR** — Список реплик, синхронизированных с лидером

- **Лидер** — Реплика, принимающая записи
- **Фолловер** — Реплика, копирующая данные с лидера

Урок 2 - Как консумеры читают с лидера

Шаг 02:

Вопрос 1

Кто отдает сообщения консумерам в Kafka?

Подсказка: Подумайте, какая реплика является источником актуальных данных.

Ответ:

- **Лидер партиции**
- Фолловер
- Продюсер
- Консумер

Шаг 03:

Вопрос 2

Почему консумеры читают только с лидера?

Подсказка: Подумайте о последовательности и согласованности данных.

Ответ:

- **Чтобы гарантировать последовательное чтение актуальных сообщений**
- Чтобы снизить нагрузку на фолловеры
- Чтобы ускорить запись продюсера
- Чтобы хранить все данные локально

Шаг 04:

Вопрос 3

Что произойдет, если лидер недоступен и консумеры пытаются читать данные?

Подсказка: Подумайте о выборе нового лидера и механизме отказоустойчивости.

Ответ:

- **Консумеры перенаправятся на нового лидера после его выбора**
- Чтение произойдет с любого фолловера
- Данные будут потеряны
- Консумеры приостановят работу до восстановления старого лидера

Шаг 05:

Вопрос 4 Расположите шаги чтения данных консумером в правильном порядке:

1. Консумер отправляет запрос лидеру
2. Лидер читает данные из сегмента
3. Лидер отправляет данные консумеру
4. Консумер подтверждает получение сообщений

 **Подсказка:**

Сначала запрос, затем чтение, отправка и подтверждение.

Ответ:

- Консумер отправляет запрос лидеру
- Лидер читает данные из сегмента
- Лидер отправляет данные консумеру
- Консумер подтверждает получение сообщений

🔗 **Шаг 06:**

Вопрос 5

Почему чтение только с лидера критично для консумеров?

Подсказка: Подумайте о согласованности и точности данных.

Ответ:

- **Чтобы избежать чтения устаревших или несогласованных данных**
- Чтобы снизить нагрузку на фолловеры
- Чтобы ускорить запись продюсера
- Чтобы хранить все данные локально

🔗 **Шаг 07:**

Вопрос 6

Сопоставьте участников системы с их ролью при чтении данных:

Ответ:

- Лидер
- Фолловер
- Консумер

Подсказка: Подумайте, кто отдает данные, кто копирует, кто получает сообщения.

- **Лидер** — Отдает актуальные сообщения консумерам
- **Фолловер** — Синхронизирует данные с лидером
- **Консумер** — Читает сообщения через лидера

🔗 **Шаг 08:**

Вопрос 7

Что произойдет, если консумер попытается читать с устаревшего фолловера?

Подсказка: Подумайте о согласованности данных и ISR.

Ответ:

- **Консумер получит устаревшие или неполные данные**
- Прочитает актуальные данные
- Система перенаправит автоматически на лидера
- Фолловер станет лидером автоматически

🔗 **Шаг 09:**

Вопрос 8 Расположите события при чтении данных консумером с лидера:

1. Консумер отправляет запрос
2. Лидер читает данные из сегмента
3. Лидер отправляет данные консумеру

4. Консумер подтверждает получение

💡 **Подсказка:**

Сначала запрос, потом чтение, отправка и подтверждение.

Ответ:

- Консумер отправляет запрос
- Лидер читает данные из сегмента
- Лидер отправляет данные консумеру
- Консумер подтверждает получение

💡 **Шаг 10:**

Вопрос 9

Сопоставьте термин с функцией при чтении сообщений:

Ответ:

- ISR
- Лидер
- Фолловер

Подсказка: Подумайте о синхронизированных репликах и источнике чтения.

- **ISR** — Список реплик, синхронизированных с лидером
- **Лидер** — Отдает актуальные данные консумерам
- **Фолловер** — Копирует данные с лидера

💡 **Шаг 11:**

Вопрос 10

Что критично для корректного чтения консумерами в Kafka?

Подсказка: Подумайте о согласованности и актуальности данных.

Ответ:

- **Наличие актуального лидера и синхронизированных фолловеров**
- Чтение напрямую с любого фолловера
- Игнорирование последовательности сообщений
- Хранение сообщений только на консумере

📖 **Урок 3 - Практическое задание симуляция сбоя брокера**

💡 **Шаг 02:**

Вопрос 1

Что происходит, когда лидер партиции выходит из строя?

Подсказка: Подумайте о роли ISR и выборе нового лидера.

Ответ:

- **Выбирается новый лидер из списка синхронизированных реплик (ISR)**
- Все фолловеры становятся лидерами одновременно
- Данные теряются без возможности восстановления
- Консумеры продолжают читать с недоступного лидера

Шаг 03:

Вопрос 2

Почему Kafka использует ISR для выбора нового лидера?

Подсказка: Подумайте о том, как гарантируется согласованность данных.

Ответ:

- Чтобы новый лидер имел все актуальные данные и не было потерь сообщений
- Чтобы ускорить выбор лидера без учета данных
- Чтобы консумеры могли читать напрямую с любого фолловера
- Чтобы фолловеры перестали синхронизироваться

Шаг 04:

Вопрос 3

Что произойдет с консумерами во время выбора нового лидера?

Подсказка: Подумайте о кратковременной недоступности данных.

Ответ:

- Они временно не смогут читать сообщения, пока лидер не выбран
- Могут читать данные с любого фолловера без ожидания
- Процесс чтения продолжится без паузы
- Сбой полностью остановит работу Kafka

Шаг 05:

Вопрос 4 Расположите действия Kafka при сбое лидера в правильном порядке:

1. Детектирование недоступности лидера
2. Выбор нового лидера из ISR
3. Обновление метаданных у всех брокеров
4. Консумеры перенаправляются к новому лидеру

 **Подсказка:**

Сначала обнаруживаем проблему, затем выбираем нового лидера, обновляем метаданные и продолжаем чтение.

Ответ:

- Детектирование недоступности лидера
- Выбор нового лидера из ISR
- Обновление метаданных у всех брокеров
- Консумеры перенаправляются к новому лидеру

Шаг 06:

Вопрос 5

Почему важно, чтобы новый лидер был в ISR?

Подсказка: Подумайте о потерях данных.

Ответ:

- Чтобы гарантировать, что все сообщения уже записаны и не потеряются
- Чтобы ускорить выбор лидера без учета данных
- Чтобы фолловеры перестали синхронизироваться

- Чтобы консумеры читали данные напрямую с любого брокера

Шаг 07:

Вопрос 6

Сопоставьте элементы системы с их ролью при сбое лидера:

Ответ:

- Лидер
- Фолловер
- ISR

Подсказка: Подумайте, кто отдает данные, кто копирует, кто участвует в выборе нового лидера.

- **Лидер** — Отдает актуальные данные консумерам до сбоя
- **Фолловер** — Копирует данные с лидера
- **ISR** — Список реплик, синхронизированных с лидером, участвующих в выборе нового лидера

Шаг 08:

Вопрос 7

Что происходит с непросинхронизированными фолловерами при выборе нового лидера?

Подсказка: Подумайте о безопасности данных.

Ответ:

- **Они не могут быть выбраны лидером, чтобы не потерять данные**
- Мгновенно становятся лидерами
- Консумеры читают с них без ожидания синхронизации
- Они удаляются из кластера

Шаг 09:

Вопрос 8 Расположите шаги восстановления работы брокера после сбоя:

1. Брокер перезапускается
2. Фолловеры синхронизируют данные с новым лидером
3. Обновление ISR
4. Консумеры продолжают чтение

 **Подсказка:**

Сначала брокер стартует, синхронизация, обновление ISR, потом консумеры.

Ответ:

- Брокер перезапускается
- Фолловеры синхронизируют данные с новым лидером
- Обновление ISR
- Консумеры продолжают чтение

Шаг 10:

Вопрос 9

Сопоставьте термины с их функцией при сбое брокера:

Ответ:

- Лидер
- Фолловер
- ISR

Подсказка: Подумайте, кто участвует в выборе нового лидера и синхронизации данных.

- **Лидер** — Отдает актуальные данные до сбоя
- **Фолловер** — Синхронизирует данные с лидера
- **ISR** — Список реплик, готовых стать лидером

Урок 4 - Мини-тест «Что будет, если лидер упал»

Шаг 02:

Вопрос 1

Что произойдет с партицией, если её лидер внезапно выйдет из строя?

Подсказка: Подумайте о роли ISR.

Ответ:

- **Выбирается новый лидер из списка синхронизированных реплик (ISR)**
- Все фолловеры становятся лидерами одновременно
- Партиция перестает принимать записи навсегда
- Консумеры продолжают читать с недоступного лидера

Шаг 03:

Вопрос 2

Почему Kafka использует ISR для выбора нового лидера?

Подсказка: Подумайте о согласованности данных и их безопасности.

Ответ:

- **Чтобы новый лидер имел все актуальные данные и не потерялись сообщения**
- Чтобы ускорить выбор лидера без учета данных
- Чтобы консумеры могли читать напрямую с любого фолловера
- Чтобы фолловеры перестали синхронизироваться

Шаг 04:

Вопрос 3

Что происходит с консумерами, пока выбирается новый лидер?

Подсказка: Подумайте о кратковременной недоступности данных.

Ответ:

- **Они временно не могут читать сообщения, пока лидер не выбран**
- Могут читать данные с любого фолловера без ожидания
- Чтение продолжается без паузы
- Сбой полностью останавливает работу кластера

Шаг 05:

Вопрос 4 Расположите шаги Kafka при сбое лидера в правильном порядке:

1. Детектирование недоступности лидера

2. Выбор нового лидера из ISR
3. Обновление метаданных у всех брокеров
4. Консумеры перенаправляются к новому лидеру

🔔 Подсказка:

Сначала обнаруживаем проблему, затем выбираем лидера, обновляем метаданные и продолжаем чтение.

Ответ:

- Детектирование недоступности лидера
- Выбор нового лидера из ISR
- Обновление метаданных у всех брокеров
- Консумеры перенаправляются к новому лидеру

🔔 Шаг 06:

Вопрос 5

Почему новый лидер выбирается только из ISR?

Подсказка: Подумайте о сохранности данных.

Ответ:

- **Чтобы гарантировать, что все сообщения уже записаны и не потеряются**
- Чтобы ускорить выбор лидера без учета данных
- Чтобы консумеры читали с любого брокера
- Чтобы фолловеры перестали синхронизироваться

🔔 Шаг 07:

Вопрос 6

Сопоставьте элементы системы с их ролью при сбое лидера:

Ответ:

- Лидер
- Фолловер
- ISR

Подсказка: Кто отдаёт данные, кто копирует, кто участвует в выборе нового лидера?

- **Лидер** — Отдает актуальные данные консумерам до сбоя
- **Фолловер** — Копирует данные с лидера
- **ISR** — Список реплик, синхронизированных с лидером, участвующих в выборе нового лидера

🔔 Шаг 08:

Вопрос 7

Что происходит с фолловерами вне ISR при выборе нового лидера?

Подсказка: Подумайте о безопасности данных.

Ответ:

- **Они не могут быть выбраны лидером, чтобы не потерять данные**
- Сразу становятся лидерами
- Консумеры читают с них без ожидания синхронизации
- Удаляются из кластера

Шаг 09:

Вопрос 8 Расположите шаги восстановления брокера после сбоя:

1. Брокер перезапускается
2. Фолловеры синхронизируют данные с новым лидером
3. Обновление ISR
4. Консумеры продолжают чтение

Подсказка:

Сначала брокер стартует, синхронизация, обновление ISR, потом консумеры.

Ответ:

- Брокер перезапускается
- Фолловеры синхронизируют данные с новым лидером
- Обновление ISR
- Консумеры продолжают чтение

Шаг 10:

Вопрос 9

Сопоставьте термины с их функцией при сбое брокера:

Ответ:

- Лидер
- Фолловер
- ISR

Подсказка: Кто участвует в выборе нового лидера и синхронизации данных?

- **Лидер** — Отдает актуальные данные до сбоя
- **Фолловер** — Синхронизирует данные с лидера
- **ISR** — Список реплик, готовых стать лидером

Шаг 11:

Вопрос 10

Что критично для корректного выбора нового лидера при сбое?

Подсказка: Подумайте о согласованности и безопасности данных.

Ответ:

- **Наличие актуальных реплик в ISR, готовых стать лидером**
- Случайный выбор любого фолловера
- Игнорирование данных на фолловерах
- Выбор нового лидера без метаданных

Модуль 5 - Смещения (Offsets) и чтение сообщений

Урок 1 - Смещение — это «номер страницы» в логе

Шаг 02:

Вопрос 1

Что обозначает offset в Kafka?

Подсказка: Подумайте, как система помнит, какие сообщения уже прочитаны.

Ответ:

- **Номер позиции сообщения в топике**
- Содержание сообщения
- Время отправки сообщения
- ID консумера

🔗 **Шаг 03:**

Вопрос 2

Как консумер использует offset?

Подсказка: Подумайте о том, как консумер понимает, с какого сообщения продолжить чтение.

Ответ:

- **Читает сообщения начиная с определённого offset**
- Меняет содержимое сообщения
- Удаляет сообщения из топика
- Отправляет offset обратно продюсеру

🔗 **Шаг 04:**

Вопрос 3

Что произойдёт, если консумер потеряет offset?

Подсказка: Подумайте о том, какие сообщения он сможет прочитать снова или пропустить.

Ответ:

- **Он может прочитать одни сообщения повторно или пропустить некоторые**
- Сообщения исчезнут из топика
- Offset автоматически восстанавливается
- Все сообщения станут недоступны

🔗 **Шаг 05:**

Вопрос 4 Расположите шаги чтения сообщений консумером по offset:

1. Консумер запрашивает сообщения с топика
2. Консумер получает пакет сообщений
3. Консумер обрабатывает сообщения
4. Консумер сохраняет текущий offset

🔗 **Подсказка:**

Сначала читаем → обрабатываем → фиксируем offset.

Ответ:

- Консумер запрашивает сообщения с топика
- Консумер получает пакет сообщений
- Консумер обрабатывает сообщения
- Консумер сохраняет текущий offset

🔗 **Шаг 06:**

Вопрос 5

Что такое committed offset?

Подсказка: Подумайте о том, какой offset считается «официально прочитанным» консумером.

Ответ:

- **Последний offset, который консумер сохранил в брокере**
- Offset следующего сообщения
- Offset первого сообщения в топике
- Случайное число

🔗 **Шаг 07:**

Вопрос 6

Сопоставьте термин с его определением:

Ответ:

- Offset
- Committed offset
- Consumer group
- Partition

Подсказка: Подумайте, кто и как отслеживает сообщения.

- **Offset** — Номер сообщения в партиции
- **Committed offset** — Последний offset, зафиксированный консумером
- **Consumer group** — Группа консумеров, совместно читающих топик
- **Partition** — Часть топика, содержащая последовательность сообщений

🔗 **Шаг 08:**

Вопрос 7

Зачем важно отслеживать offset при обработке сообщений?

Подсказка: Подумайте о надежности и повторной обработке.

Ответ:

- **Чтобы не пропустить и не обработать повторно сообщения**
- Чтобы ускорить продюсер
- Чтобы удалить топик
- Чтобы консумер удалял сообщения

🔗 **Шаг 09:**

Вопрос 8 Расположите действия консумера по обработке пакета сообщений и фиксации offset:

1. Консумер получает сообщения
2. Обрабатывает каждое сообщение
3. Фиксирует последний обработанный offset
4. Готов к следующему запросу сообщений

🔗 **Подсказка:**

Получение → обработка → сохранение offset → подготовка к следующему чтению.

Ответ:

- Консумер получает сообщения
- Обрабатывает каждое сообщение
- Фиксирует последний обработанный offset

- Готов к следующему запросу сообщений

Шаг 10:

Вопрос 9

Сопоставьте сценарий с результатом:

Ответ:

- Консумер падает до фиксации offset
- Консумер фиксирует offset после обработки
- Сообщение в топике еще не прочитано

Подсказка: Подумайте о повторной обработке и пропуске сообщений.

- **Консумер падает до фиксации offset** — Сообщения будут обработаны повторно после перезапуска
- **Консумер фиксирует offset после обработки** — Следующий запуск пропускает уже обработанные сообщения
- **Сообщение в топике еще не прочитано** — Можно начать обработку с этого offset

Урок 2 - Авто- и ручное управление смещениями

Шаг 02:

Вопрос 1

Что означает автоматическое управление offset в Kafka?

Подсказка: Подумайте, кто сохраняет offset без вмешательства консумера.

Ответ:

- **Kafka автоматически сохраняет offset после обработки сообщений**
- Консумер должен сам каждый раз вручную фиксировать offset
- Offset не сохраняется вовсе
- Offset обновляется только продюсером

Шаг 03:

Вопрос 2

Что делает ручное управление offset?

Подсказка: Подумайте, кто решает, когда offset фиксировать.

Ответ:

- **Консумер сам фиксирует offset в нужный момент**
- Kafka фиксирует offset автоматически
- Offset не используется
- Продюсер фиксирует offset вручную

Шаг 04:

Вопрос 3

Какой плюс ручного управления offset?

Подсказка: Подумайте о контроле над повторной обработкой сообщений.

Ответ:

- **Можно точно управлять, какие сообщения считать обработанными**

- Сообщения обрабатываются быстрее
- Offset фиксируется автоматически
- Сообщения удаляются из топика

🔗 Шаг 05:

Вопрос 4 Расположите шаги при ручном управлении offset:

1. Консумер получает сообщения
2. Обрабатывает сообщения
3. Ручная фиксация offset
4. Готов к следующему чтению

🔗 Подсказка:

Сначала получаем → обрабатываем → фиксируем offset → продолжаем чтение.

Ответ:

- Консумер получает сообщения
- Обрабатывает сообщения
- Ручная фиксация offset
- Готов к следующему чтению

🔗 Шаг 06:

Вопрос 5

Что может произойти при автоматическом управлении offset, если консумер падает до фиксации?

Подсказка: Подумайте, какие сообщения могут быть обработаны повторно.

Ответ:

- **Некоторые сообщения могут быть обработаны дважды**
- Все сообщения теряются
- Ничего не произойдет
- Сообщения автоматически удаляются из топика

🔗 Шаг 07:

Вопрос 6

Сопоставьте тип управления offset с особенностью:

Ответ:

- Автоматическое управление
- Ручное управление

Подсказка: Подумайте о контроле консумера над фиксацией offset.

- **Автоматическое управление** — Kafka фиксирует offset сама после интервала
- **Ручное управление** — Консумер сам решает, когда сохранить offset

🔗 Шаг 08:

Вопрос 7

Когда лучше использовать ручное управление offset?

Подсказка: Подумайте о ситуациях с критически важной обработкой данных.

Ответ:

- **Когда важно точно контролировать, какие сообщения обработаны**
- Когда важна только скорость обработки
- Когда offset не нужен
- Когда топик содержит только одно сообщение

🔗 Шаг 09:

Вопрос 8 Расположите действия при автоматическом управлении offset:

1. Консумер получает сообщения
2. Обрабатывает сообщения
3. Kafka автоматически фиксирует offset через интервал
4. Готов к следующему пакету сообщений

🔗 Подсказка:

Получение → обработка → автофиксация offset → продолжение чтения.

Ответ:

- Консумер получает сообщения
- Обрабатывает сообщения
- Kafka автоматически фиксирует offset через интервал
- Готов к следующему пакету сообщений

🔗 Шаг 10:

Вопрос 9

Сопоставьте сценарий с результатом:

Ответ:

- Ручное управление, консумер фиксирует offset после обработки
- Автоматическое управление, консумер падает до автофиксации

Подсказка: Подумайте о повторной обработке сообщений.

- **Ручное управление, консумер фиксирует offset после обработки** — Следующий запуск пропускает уже обработанные сообщения
- **Автоматическое управление, консумер падает до автофиксации** — Некоторые сообщения будут обработаны повторно

🔗 Шаг 11:

Вопрос 10

Как обеспечить надежность обработки сообщений при использовании авто- и ручного управления offset?

Подсказка: Подумайте о комбинировании контроля и резервирования offset.

Ответ:

- **Использовать ручное управление для критичных сообщений и авто для остального**
- Никогда не фиксировать offset
- Полностью полагаться на автофиксацию без контроля
- Удалять топик после чтения

Урок 3 - Практика чтения сообщений консьюмером с разными смещениями

Шаг 02:

Вопрос 1

Консьюмер Kafka только начал читать топик. С какого offset он должен начать?

Подсказка: Подумайте, есть ли разница между новыми и старыми сообщениями.

Ответ:

- **С начала топика или с последнего сообщения, в зависимости от настроек консьюмера**
- С случайного сообщения в топике
- С середины топика всегда
- Только с самого последнего сообщения

Шаг 03:

Вопрос 2

Консьюмеру нужно прочитать сообщения с конкретного offset. Как это сделать?

Подсказка: Подумайте про методы установки позиции чтения в Kafka.

Ответ:

- **Использовать метод `seek()` для установки нужного offset**
- Пропустить сообщения до нужного номера вручную
- Перезапустить консьюмера до достижения offset
- Kafka сама выберет offset случайно

Шаг 04:

Вопрос 3

Сопоставьте термины с поведением консьюмера:

Ответ:

- Чтение с `earliest`
 - Чтение с `latest`
 - `seek(offset)`
 - `Auto-commit offset`
1. Начало чтения с самого первого сообщения в партиции
 2. Начало чтения с последнего сообщения, игнорируя старые
 3. Перейти к конкретному offset для чтения
 4. Kafka сохраняет offset автоматически через интервал

Подсказка:

Подумайте, как выбрать, что читать: старые сообщения, новые или конкретный номер.

- **Чтение с `earliest`** — Начало чтения с самого первого сообщения в партиции
- **Чтение с `latest`** — Начало чтения с последнего сообщения, игнорируя старые
- **`seek(offset)`** — Перейти к конкретному offset для чтения
- **`Auto-commit offset`** — Kafka сохраняет offset автоматически через интервал

🔗 Шаг 05:

Вопрос 4 Расположите шаги чтения сообщений с разными offset:

1. Установить нужный offset (seek)
2. Запросить сообщения у брокера
3. Обработать полученные сообщения
4. Сохранить offset после успешной обработки

🔗 Подсказка:

Сначала устанавливаем позицию, потом читаем, обрабатываем и фиксируем прогресс.

Ответ:

- Установить нужный offset (seek)
- Запросить сообщения у брокера
- Обработать полученные сообщения
- Сохранить offset после успешной обработки

🔗 Шаг 06:

Вопрос 5

Консьюмер пропускает сообщения при автоматическом коммите. В чем причина?

Подсказка: Подумайте, когда Kafka сохраняет offset при auto-commit.

Ответ:

- **Offset сохраняется до обработки сообщения**
- **При сбое консьюмер может пропустить сообщение**
- Kafka теряет все сообщения при restart
- Сообщения удаляются сразу после публикации

🔗 Шаг 07:

Вопрос 6

Сопоставьте жизненные ситуации с типом commit offset:

Ответ:

- Обработка финансовых транзакций
 - Сбор логов для аналитики
 - Высоконагруженный поток сообщений
 - Экспериментальный консьюмер
1. Manual commit offset для точности
 2. Auto-commit offset с интервалом
 3. Чтение нескольких партиций параллельно
 4. Можно использовать auto-commit без страха потерь

🔗 Подсказка:

Подумайте, где важна точность, а где важна скорость и простота.

- **Обработка финансовых транзакций** — Manual commit offset для точности
- **Сбор логов для аналитики** — Auto-commit offset с интервалом
- **Высоконагруженный поток сообщений** — Чтение нескольких партиций параллельно
- **Экспериментальный консьюмер** — Можно использовать auto-commit без страха потерь

Шаг 08:

Вопрос 7

Почему стоит использовать seek при повторном чтении старых сообщений?

Подсказка: Подумайте, как получить конкретное сообщение без потери данных.

Ответ:

- Чтобы установить точный offset и заново прочитать нужные сообщения
- Чтобы пропустить все старые сообщения
- Чтобы случайно выбрать сообщение
- Чтобы удалить старые сообщения

Шаг 09:

Вопрос 8 Расположите шаги обработки ошибок при чтении сообщений с разными offset:

1. Консьюмер получает сообщение
2. Обнаруживает ошибку
3. Сохраняет offset только после успешной обработки
4. Повторная обработка при сбое

 **Подсказка:**

Сначала читаем, проверяем, сохраняем offset после успеха, иначе повторяем.

Ответ:

- Консьюмер получает сообщение
- Обнаруживает ошибку
- Сохраняет offset только после успешной обработки
- Повторная обработка при сбое

Шаг 10:

Вопрос 10

Сопоставьте жизненные ситуации с подходами управления offset:

Ответ:

- Критичные транзакции
 - Сбор логов
 - Высоконагруженный поток
 - Тестовый консьюмер
1. Manual commit offset после обработки для точности
 2. Auto-commit offset с интервалом для простоты
 3. Чтение нескольких партиций параллельно для скорости
 4. Auto-commit и не бояться потерь

 **Подсказка:**

Подумайте, где важна точность, где скорость и удобство.

- **Критичные транзакции** — Manual commit offset после обработки для точности
- **Сбор логов** — Auto-commit offset с интервалом для простоты
- **Высоконагруженный поток** — Чтение нескольких партиций параллельно для скорости
- **Тестовый консьюмер** — Auto-commit и не бояться потерь

Урок 4 - Мини-тест «Что такое offset»

Шаг 02:

Вопрос 1

Что такое offset в Kafka?

Подсказка: Подумайте, как Kafka отслеживает позицию чтения сообщений в топике.

Ответ:

- **Уникальный номер сообщения в партиции, указывающий его позицию**
- Размер сообщения в байтах
- Имя топика, откуда взято сообщение
- Время публикации сообщения

Шаг 03:

Вопрос 2

Почему offset важен для консюмера?

Подсказка: Подумайте, как избежать повторного чтения или пропуска сообщений.

Ответ:

- **Позволяет консумеру помнить, какие сообщения он уже прочитал**
- **Позволяет возобновить чтение после сбоя**
- Определяет размер очереди сообщений
- Позволяет брокеру удалять старые топики

Шаг 04:

Вопрос 3

Сопоставьте типы чтения с поведением консюмера:

Ответ:

- earliest
 - latest
 - seek(offset)
 - auto-commit
1. Чтение с начала топика
 2. Чтение с конца, новые сообщения
 3. Перейти к конкретному offset
 4. Автоматическое сохранение offset

Подсказка:

Подумайте, где важно начать с начала, с конца или с конкретного места.

- **earliest** — Чтение с начала топика
- **latest** — Чтение с конца, новые сообщения
- **seek(offset)** — Перейти к конкретному offset
- **auto-commit** — Автоматическое сохранение offset

🔔 Шаг 05:

Вопрос 4 Расположите шаги корректного чтения сообщений с учетом offset:

1. Консьюмер получает сообщение
2. Обрабатывает сообщение
3. Сохраняет offset после успешной обработки
4. При сбое использует сохраненный offset для повторного чтения

🔔 Подсказка:

Сначала читаем, затем обрабатываем, фиксируем offset, чтобы избежать потерь или дубликатов.

Ответ:

- Консьюмер получает сообщение
- Обрабатывает сообщение
- Сохраняет offset после успешной обработки
- При сбое использует сохраненный offset для повторного чтения

🔔 Шаг 06:

Вопрос 5

Что произойдет, если консьюмер не сохраняет offset вручную при auto-commit выключен?

Подсказка: Подумайте, как Kafka узнает, что сообщение уже прочитано.

Ответ:

- **Сообщения могут быть прочитаны повторно после перезапуска консьюмера**
- **Можно потерять точку продолжения чтения**
- Kafka сама исправит offset автоматически
- Сообщения удалятся из топика

🔔 Шаг 07:

Вопрос 6

Сопоставьте жизненные ситуации с типом offset:

Ответ:

- Аналитика логов
 - Финансовые транзакции
 - Тестовый консьюмер
 - Высоконагруженный поток
1. auto-commit
 2. manual commit
 3. Можно использовать earliest
 4. Чтение нескольких партиций параллельно

🔔 Подсказка:

Подумайте, где важна скорость, а где точность.

- **Аналитика логов** — auto-commit
- **Финансовые транзакции** — manual commit
- **Тестовый консьюмер** — Можно использовать earliest
- **Высоконагруженный поток** — Чтение нескольких партиций параллельно

Шаг 08:

Вопрос 7

Зачем использовать seek() для offset?

Подсказка: Подумайте, когда нужно заново прочитать определённые сообщения.

Ответ:

- Чтобы установить точный offset и прочитать нужные сообщения повторно
- Чтобы удалить сообщения из топика
- Чтобы пропустить все сообщения автоматически
- Чтобы изменить значение offset на случайное

Шаг 09:

Вопрос 8 Расположите действия при обработке ошибок с offset:

1. Консьюмер получает сообщение
2. Обнаруживает ошибку
3. Не сохраняет offset
4. Повторная обработка сообщения после исправления ошибки

 **Подсказка:**

Сначала читаем, проверяем, не фиксируем offset при ошибке, повторяем чтение после исправления.

Ответ:

- Консьюмер получает сообщение
- Обнаруживает ошибку
- Не сохраняет offset
- Повторная обработка сообщения после исправления ошибки

Шаг 10:

Вопрос 9

Как гарантировать обработку каждого сообщения только один раз?

Подсказка: Подумайте про ручное управление offset и обработку сообщений.

Ответ:

- Использовать ручной commit offset после успешной обработки сообщения
- Полагаться только на auto-commit
- Пропускать сообщения при ошибках
- Читать только новые сообщения

Шаг 11:

Вопрос 10

Что произойдет, если offset вручную установлен на старое сообщение и консьюмер снова его прочитает?

Подсказка: Подумайте о повторной обработке сообщений и возможных дубликатах.

Ответ:

- Сообщение будет прочитано заново
- Если нет идемпотентной обработки, возможны дубликаты

- Kafka удалит сообщение после чтения
- Offset изменится автоматически на новое сообщение

Модуль 6 - Продюсер, консумер и внутренние механизмы

Урок 1 - Продюсер пишет в лог, консумер читает из лога

Шаг 02:

Вопрос 1

Что делает продюсер в Kafka?

Подсказка: Подумайте о том, кто отправляет сообщения в топик.

Ответ:

- **Отправляет сообщения в топик**
- Читает сообщения из топика
- Управляет брокерами
- Сохраняет offset консумеров

Шаг 03:

Вопрос 2

Что делает консумер в Kafka?

Подсказка: Подумайте о том, кто читает сообщения.

Ответ:

- **Читает сообщения из топика**
- Отправляет сообщения в топик
- Создает новые партиции
- Следит за репликацией

Шаг 04:

Вопрос 3

Почему Kafka использует логи для хранения сообщений?

Подсказка: Подумайте о последовательности и скорости чтения.

Ответ:

- **Чтобы хранить сообщения в порядке поступления и быстро отдавать консумерам**
- Чтобы шифровать данные сообщений
- Чтобы контролировать репликацию брокеров
- Чтобы уменьшить размер топика

Шаг 05:

Вопрос 4 Расположите шаги работы продюсера и консумера в правильном порядке:

1. Продюсер отправляет сообщение в топик
2. Сообщение записывается в лог партиции
3. Консумер считывает сообщение
4. Консумер обрабатывает сообщение

Подсказка:

Сначала пишем, потом лог фиксирует, затем читаем и обрабатываем.

Ответ:

- Продюсер отправляет сообщение в топик
- Сообщение записывается в лог партиции
- Консумер считывает сообщение
- Консумер обрабатывает сообщение

Шаг 06:

Вопрос 5

Что происходит, если консумер читает из лога слишком медленно?

Подсказка: Подумайте о накоплении сообщений в топике.

Ответ:

- **Сообщения накапливаются в топике, увеличивая задержку доставки**
- Kafka автоматически удаляет консумера
- Сообщения теряются сразу
- Продюсер перестает отправлять сообщения

Шаг 07:

Вопрос 6

Сопоставьте термины с их функцией:

Ответ:

- Продюсер
- Консумер
- Топик
- Лог

Подсказка: Кто пишет, кто читает и где хранятся сообщения.

- **Продюсер** — Отправляет сообщения в топик
- **Консумер** — Читает сообщения из топика
- **Топик** — Логическая группа сообщений
- **Лог** — Хранение сообщений в порядке поступления

Шаг 08:

Вопрос 7

Почему Kafka сохраняет сообщения в партициях, а не в одном большом логе?

Подсказка: Подумайте о масштабируемости и параллельном чтении.

Ответ:

- **Чтобы позволить параллельное чтение и масштабирование продюсеров и консумеров**
- Чтобы уменьшить размер топика
- Чтобы шифровать данные быстрее
- Чтобы ограничить количество брокеров

Шаг 09:

Вопрос 8 Расположите шаги обработки сообщений консумером с сохранением offset:

1. Получение сообщений из лога
2. Обработка сообщений
3. Коммит offset
4. Продолжение чтения следующей партии

Подсказка:

Чтение → обработка → сохранение позиции → продолжение чтения.

Ответ:

- Получение сообщений из лога
- Обработка сообщений
- Коммит offset
- Продолжение чтения следующей партии

Шаг 10:

Вопрос 9

Сопоставьте ситуации с действиями продюсера и консумера:

Ответ:

- Продюсер отправил много сообщений
- Консумер упал
- Консумер стартовал заново

Подсказка: Подумайте, как offset помогает консумеру продолжить работу.

- **Продюсер отправил много сообщений** — Сообщения накапливаются в логе топика
- **Консумер упал** — Offset не обновляется до последнего обработанного сообщения
- **Консумер стартовал заново** — Чтение продолжается с последнего закоммиченного offset

Шаг 11:

Вопрос 10

Что критично для корректной работы продюсера и консумера в Kafka?

Подсказка: Подумайте о порядке сообщений и надежности чтения.

Ответ:

- **Согласованное чтение и запись с использованием offset и логов**
- Игнорирование offset и чтение сообщений случайно
- Отправка сообщений без топиков
- Удаление логов после каждой записи

Урок 2 - Пакетная отправка сообщений и подтверждения (ack)

Шаг 02:

Вопрос 1

Что такое пакетная отправка сообщений в Kafka?

Подсказка: Подумайте о том, как продюсер может отправлять сразу несколько сообщений.

Ответ:

- **Отправка группы сообщений в одном пакете**
- Отправка сообщений по одному
- Удаление сообщений из лога
- Чтение сообщений консумером

Шаг 03:

Вопрос 2

Что означает подтверждение (ack) от брокера?

Подсказка: Подумайте о том, как продюсер узнаёт, что сообщение доставлено.

Ответ:

- **Брокер сообщил продюсеру, что сообщение успешно записано**
- Консумер прочитал сообщение
- Топик удалил сообщение
- Брокер заблокировал сообщение

Шаг 04:

Вопрос 3

Какая выгода пакетной отправки сообщений?

Подсказка: Подумайте о скорости и нагрузке на сеть.

Ответ:

- **Снижение нагрузки на сеть и повышение производительности**
- Увеличение времени доставки сообщений
- Повышение вероятности потери сообщений
- Изоляция консумеров от топиков

Шаг 05:

Вопрос 4 Расположите шаги пакетной отправки сообщений с подтверждением в правильном порядке:

1. Продюсер собирает несколько сообщений в пакет
2. Отправка пакета в брокер
3. Брокер записывает пакет в лог
4. Брокер отправляет ack продюсеру

Подсказка:

Сначала формируем пакет, потом отправляем, фиксируем на сервере, затем подтверждаем.

Ответ:

- Продюсер собирает несколько сообщений в пакет
- Отправка пакета в брокер
- Брокер записывает пакет в лог
- Брокер отправляет ack продюсеру

Шаг 06:

Вопрос 5

Что произойдёт, если продюсер не дожждётся ack?

Подсказка: Подумайте о надёжности доставки сообщений.

Ответ:

- **Продюсер может повторить отправку сообщений, чтобы не потерять данные**
- Сообщения гарантированно потеряются
- Конsumer сразу получит сообщения дважды
- Топик удалит все предыдущие сообщения

Шаг 07:

Вопрос 6

Сопоставьте тип ack с его поведением:

Ответ:

- `acks=0`
- `acks=1`
- `acks=all`

Подсказка: Подумайте о количестве брокеров, которые подтверждают запись.

- **`acks=0`** — Продюсер не ждёт подтверждения
- **`acks=1`** — Ждёт подтверждения только от лидера партии
- **`acks=all`** — Ждёт подтверждения от всех реплик партии

Шаг 08:

Вопрос 7

Почему важно использовать `acks=all` для критичных сообщений?

Подсказка: Подумайте о надёжности и потере данных при сбое брокера.

Ответ:

- **Это обеспечивает, что все реплики получили сообщение, минимизируя риск потери данных**
- Это ускоряет отправку сообщений
- Это уменьшает нагрузку на брокеры
- Это гарантирует, что консумер сразу получит сообщение

Шаг 09:

Вопрос 8 Расположите события при пакетной отправке сообщений с `acks=all`:

1. Продюсер собирает пакет сообщений
2. Отправка пакета лидеру партии
3. Лидер реплицирует пакет на всех репликах
4. Все реплики отправляют подтверждение лидеру
5. Лидер отправляет ack продюсеру

 **Подсказка:**

Сначала собираем пакет → лидер → реплики → ack → продюсер.

Ответ:

- Продюсер собирает пакет сообщений
- Отправка пакета лидеру партии
- Лидер реплицирует пакет на всех репликах

- Все реплики отправляют подтверждение лидеру
- Лидер отправляет ask продюсеру

🔗 Шаг 10:

Вопрос 9

Сопоставьте сценарий с результатом при пакетной отправке:

Ответ:

- Консумер упал
- Брокер не подтвердил ask
- Пакет успешно реплицирован

Подсказка: Подумайте о повторной отправке и надежности.

- **Консумер упал** — Сообщения остаются в топике, offset не обновлен
- **Брокер не подтвердил ask** — Продюсер может повторно отправить пакет
- **Пакет успешно реплицирован** — Данные надежно сохранены на всех репликах

📖 Урок 3 - Практическое задание моделируем ask-схему с карточками

🔗 Шаг 02:

Вопрос 1

Что происходит, когда продюсер передаёт карточку (сообщение) в Kafka?

Подсказка: Подумайте о том, кто отправляет информацию и куда она попадает.

Ответ:

- **Карточка добавляется в топик**
- Карточка сразу исчезает
- Карточка отправляется консумеру напрямую
- Карточка сохраняется только у продюсера

🔗 Шаг 03:

Вопрос 2

Что делает консумер, когда получает карточку с сообщением?

Подсказка: Подумайте о том, кто читает информацию из топика.

Ответ:

- **Читает карточку и обрабатывает её**
- Добавляет карточку в топик
- Игнорирует карточку
- Удаляет топик

🔗 Шаг 04:

Вопрос 3

Зачем продюсер ждёт ask после отправки карточки?

Подсказка: Подумайте, как продюсер узнаёт, что сообщение дошло.

Ответ:

- **Чтобы убедиться, что карточка успешно добавлена в топик**
- Чтобы сразу удалить карточку

- Чтобы консумер получил карточку дважды
- Чтобы топик очистился

🔮 Шаг 05:

Вопрос 4 Расположите шаги ask-схемы с карточками в правильном порядке:

1. Продюсер отправляет пакет карточек
2. Карточки добавляются в лог топика
3. Брокер отправляет ask продюсеру
4. Консумер читает карточки и обрабатывает их

🔮 Подсказка:

Сначала продюсер → лог → ask → консумер.

Ответ:

- Продюсер отправляет пакет карточек
- Карточки добавляются в лог топика
- Брокер отправляет ask продюсеру
- Консумер читает карточки и обрабатывает их

🔮 Шаг 06:

Вопрос 5

Что произойдёт, если продюсер не дожждётся ask?

Подсказка: Подумайте о повторной отправке карточки.

Ответ:

- **Продюсер может повторно отправить пакет карточек**
- Карточки гарантированно потеряются
- Консумер получит карточки дважды автоматически
- Топик удалится

🔮 Шаг 07:

Вопрос 6

Сопоставьте тип ask с поведением:

Ответ:

- `acks=0`
- `acks=1`
- `acks=all`

Подсказка: Подумайте о количестве подтверждений от брокеров.

- **`acks=0`** — Продюсер не ждёт подтверждения
- **`acks=1`** — Ждёт подтверждения только от лидера партии
- **`acks=all`** — Ждёт подтверждения от всех реплик

🔮 Шаг 08:

Вопрос 7

Почему для важных карточек лучше использовать `acks=all`?

Подсказка: Подумайте о надежности и потере данных.

Ответ:

- Это обеспечивает, что все реплики получили сообщение, минимизируя риск потери
- Это ускоряет отправку карточек
- Это уменьшает нагрузку на брокеры
- Это гарантирует мгновенное получение консумером

🔔 Шаг 09:

Вопрос 8 Расположите шаги обработки пакета карточек с acks=all:

1. Продюсер формирует пакет карточек
2. Отправка пакета лидеру партии
3. Лидер реплицирует пакет на все реплики
4. Все реплики отправляют подтверждение лидеру
5. Лидер отправляет ack продюсеру
6. Консумер получает и обрабатывает карточки

🔔 Подсказка:

Формирование → лидер → репликация → подтверждения → ack → консумер.

Ответ:

- Продюсер формирует пакет карточек
- Отправка пакета лидеру партии
- Лидер реплицирует пакет на все реплики
- Все реплики отправляют подтверждение лидеру
- Лидер отправляет ack продюсеру
- Консумер получает и обрабатывает карточки

🔔 Шаг 10:

Вопрос 9

Сопоставьте ситуацию с результатом:

Ответ:

- Консумер упал
- Брокер не отправил ack
- Пакет карточек успешно реплицирован

Подсказка: Подумайте о повторной отправке и сохранении данных.

- **Консумер упал** — Карточки остаются в топике, offset не обновлён
- **Брокер не отправил ack** — Продюсер может повторно отправить пакет
- **Пакет карточек успешно реплицирован** — Данные надёжно сохранены на всех репликах

🔔 Шаг 11:

Вопрос 10

Что критично контролировать при моделировании ack-схемы с карточками?

Подсказка: Подумайте о порядке, надёжности и повторной отправке.

Ответ:

- **Размер пакета, тип ack и обработку повторной отправки**

- Игнорирование offset консумеров
- Удаление всех карточек сразу
- Отправку карточек без топиков

Урок 4 - Итоговое задание мини-кластер Kafka на доске

Шаг 02:

Вопрос 1

Что делает продюсер в Kafka?

Подсказка: Подумайте, кто добавляет сообщения в топик.

Ответ:

- **Отправляет сообщения в топик**
- Читает сообщения из топика
- Удаляет топики
- Управляет смещениями

Шаг 03:

Вопрос 2

Что делает консумер в Kafka?

Подсказка: Подумайте, кто получает сообщения из топика для обработки.

Ответ:

- **Читает сообщения из топика**
- Создает новые топики
- Отправляет сообщения в топик
- Фиксирует смещения автоматически

Шаг 04:

Вопрос 3

Что такое мини-кластер Kafka на доске?

Подсказка: Представьте маленький рабочий кластер для тестирования.

Ответ:

- **Небольшой кластер Kafka для экспериментов и обучения**
- Полноценный боевой кластер
- Только база данных сообщений
- GUI-инструмент для продюсеров

Шаг 05:

Вопрос 4 Расположите шаги запуска мини-кластера Kafka:

1. Запуск Zookeeper
2. Запуск брокеров Kafka
3. Создание топиков
4. Запуск продюсеров и консумеров

Подсказка:

Сначала необходим координатор (Zookeeper), потом брокеры, потом топики и только затем работа с сообщениями.

Ответ:

- Запуск Zookeeper
- Запуск брокеров Kafka
- Создание топиков
- Запуск продюсеров и консумеров

Шаг 06:

Вопрос 5

Для чего нужны партиции в топике Kafka?

Подсказка: Подумайте о масштабировании и порядке сообщений.

Ответ:

- **Позволяют распределять сообщения между брокерами и обеспечивать параллельную обработку**
- Служат для резервного копирования Zookeeper
- Фиксируют offset автоматически
- Удаляют старые сообщения

Шаг 07:

Вопрос 6

Сопоставьте компонент мини-кластера с его функцией:

Ответ:

- Zookeeper
- Брокер Kafka
- Продюсер
- Конsumer

Подсказка: Подумайте, кто управляет координацией, кто отправляет, кто получает.

- **Zookeeper** — Координирует брокеры и хранит метаданные
- **Брокер Kafka** — Хранит и распределяет сообщения в топиках
- **Продюсер** — Отправляет сообщения в топики
- **Консумер** — Читает сообщения из топиков

Шаг 08:

Вопрос 7

Что произойдет, если консумер начнет читать топик с начала без сохраненного offset?

Подсказка: Подумайте, откуда он возьмет сообщения.

Ответ:

- **Консумер прочитает все сообщения с самого начала топика**
- Ничего не прочитает
- Прочитает только последние сообщения
- Kafka выдаст ошибку

Шаг 09:

Вопрос 8 Расположите действия продюсера при отправке сообщения:

1. Создание сообщения

2. Выбор топика
3. Отправка сообщения брокеру
4. Получение подтверждения

🔗 **Подсказка:**

Создали → выбрали топик → отправили → дождались ack.

Ответ:

- Создание сообщения
- Выбор топика
- Отправка сообщения брокеру
- Получение подтверждения

🔗 **Шаг 10:**

Вопрос 9

Сопоставьте тип сообщения с его обработкой:

Ответ:

- Синхронная отправка
- Асинхронная отправка

Подсказка: Подумайте о том, когда консумер ждет подтверждения от брокера.

- **Синхронная отправка** — Продюсер ждет подтверждения брокера перед продолжением
- **Асинхронная отправка** — Продюсер отправляет и продолжает работу без ожидания

🔗 **Шаг 11:**

Вопрос 10

Как проверить, что мини-кластер Kafka работает корректно?

Подсказка: Подумайте о тестовой отправке и чтении сообщений.

Ответ:

- **Отправить тестовое сообщение продюсером и прочитать его консумером**
- Перезапустить брокеры без тестов
- Удалить топик и проверить логи
- Проверить версию Kafka

📁 **Модуль 7 - Мониторинг и отладка**

📖 **Урок 1 - Проверка реплик, смещений и задержек**

🔗 **Шаг 02:**

Вопрос 1

Представь, что программист отправил сообщение в Kafka-топик. Что гарантирует репликация?

Подсказка: Репликация нужна не ради скорости, а ради надёжности.

Ответ:

- **Сообщение сохранится даже при падении одного брокера**
- Сообщение всегда читается быстрее
- Сообщение превращается в несколько копий для ускорения чтения
- Сообщение исчезает после обработки

🔮 Шаг 03:

Вопрос 2

Ты смотришь на задержку (lag) у consumer group. Что означает высокий lag?

Подсказка: Lag — это всегда разница между producer и consumer.

Ответ:

- **Консьюмеры не успевают читать сообщения с партиций**
- Брокер не принимает новые сообщения
- Топик не настроен на репликацию
- Производитель перестал писать данные

🔮 Шаг 04:

Вопрос 3

Сопоставь понятия Kafka с их объяснением:

🔮 **Подсказка:**

Вспомни: offset — это позиция, а lag — отставание.

Ответ:

- **Offset** — Позиция сообщения внутри партиции
- **Lag** — Разница между последним offset и прочитанным
- **Replica** — Копия партиции на другом брокере
- **Leader** — Основная партиция, с которой работают клиенты

🔮 Шаг 05:

Вопрос 4

При отладке кластера Kafka заметили, что одна из реплик сильно отстаёт от лидера. Как это называется?

Подсказка: Реплики должны быть синхронизированы.

Ответ:

- **Out-of-sync replica**
- Offset mismatch
- Replica crash
- Lag leader

🔮 Шаг 06:

Вопрос 5

Расположи шаги мониторинга задержек в правильном порядке:

Ответ:

- Анализ причин отставания
- Сбор метрик lag через инструменты мониторинга
- Принятие решения: увеличить число консьюмеров или оптимизировать код
- Настройка алертов при превышении порога lag

🔮 **Подсказка:**

Сначала метрики, потом алерты, далее анализ и действия.

- Сбор метрик lag через инструменты мониторинга

- Настройка алертов при превышении порога lag
- Анализ причин отставания
- Принятие решения: увеличить число консьюмеров или оптимизировать код

🔗 Шаг 07:

Вопрос 6

DevOps настраивает мониторинг Kafka. Какой показатель важнее всего для контроля отставания консьюмеров?

Подсказка: Этот показатель прямо говорит об отставании.

Ответ:

- **Consumer lag**
- Replication factor
- Disk usage
- Producer throughput

🔗 Шаг 08:

Вопрос 7

Сопоставь инструменты мониторинга Kafka с их назначением:

🔗 **Подсказка:**

Вспомни: Prometheus собирает, Grafana показывает.

Ответ:

- **Prometheus** — Сбор метрик с брокеров Kafka
- **Grafana** — Визуализация данных и дашборды
- **JMX** — Доступ к внутренним метрикам брокера
- **Kafka Manager** — Управление и наблюдение за кластерами Kafka

🔗 Шаг 09:

Вопрос 8

Что должен сделать разработчик, если consumer lag стабильно растёт?

Подсказка: Решение всегда связано с балансом ресурсов.

Ответ:

- **Увеличить количество консьюмеров в группе**
- **Оптимизировать обработку сообщений**
- Снизить replication factor
- Удалить старые топики

🔗 Шаг 10:

Вопрос 9

Расположи шаги при отладке упавшего брокера Kafka:

Ответ:

- Анализ логов брокера
- Проверка состояния ZooKeeper/контроллера
- Запуск брокера заново
- Проверка восстановления лидерства и реплик

🔔 Подсказка:

Сначала читаем логи, потом проверяем зависимые сервисы, потом перезапуск и финальная проверка.

- Анализ логов брокера
- Проверка состояния ZooKeeper/контроллера
- Запуск брокера заново
- Проверка восстановления лидерства и реплик

🔔 Шаг 11:

Вопрос 10

Сопоставь типичную проблему в Kafka и её возможное решение:

🔔 Подсказка:

Думай как инженер поддержки: для каждой проблемы — свой инструмент.

Ответ:

- **Высокий consumer lag** — Увеличить число консьюмеров или ускорить обработку
- **Out-of-sync replica** — Проверить сетевую нагрузку и здоровье диска
- **Слишком много мелких сообщений** — Использовать batching у продюсера
- **Заполнение диска брокера** — Настроить политику retention и очистку

📖 Урок 2 - Простая визуализация кто лидер, кто отстаёт

🔔 Шаг 02:

Вопрос 1

Представь, что в Kafka есть несколько реплик одного топика. Как понять, кто из них лидер?

Подсказка: Лидер — тот, кто отвечает за запись и чтение, остальные догоняют.

Ответ:

- **Это реплика, которая принимает все записи и раздаёт их читателям**
- Это случайная реплика, которую выбирает продюсер
- Это самая старая реплика в кластере
- Это реплика, где хранится больше всего данных

🔔 Шаг 03:

Вопрос 2

При мониторинге Kafka ты видишь отстающую реплику. Что это значит?

Подсказка: Отставание связано с синхронизацией.

Ответ:

- **Эта реплика не успевает копировать данные от лидера**
- Эта реплика назначена новой ведущей
- Эта реплика хранит другие топика
- Эта реплика отключилась навсегда

🔔 Шаг 04:

Вопрос 3

Сопоставь термин и его описание:

🔔 Подсказка:

Подумай: лидер, реплика, отставание, ISR.

Ответ:

- **Лидер** — Реплика, принимающая записи и выдающая чтения.
- **Фолловер** — Реплика, которая догоняет лидера.
- **Отставание (lag)** — Разница между позицией лидера и фолловера.
- **ISR** — Список реплик, которые синхронизированы с лидером.

🔗 Шаг 05:

Вопрос 4

Ты хочешь понять, кто лидер и кто отстаёт. Расставь шаги мониторинга:

🔗 **Подсказка:**

Сначала смотри на список реплик, потом определяй лидера и ISR.

Ответ:

- Посмотреть список реплик партии
- Определить лидера
- Сравнить смещения лидера и фолловеров
- Составить список ISR

🔗 Шаг 06:

Вопрос 5

Если лидер выходит из строя, что делает Kafka?

Подсказка: Важно, чтобы данные не потерялись.

Ответ:

- **Выбирает нового лидера из ISR**
- Удаляет все данные топика
- Переводит кластер в read-only режим
- Ожидает ручного вмешательства

🔗 Шаг 07:

Вопрос 6

Сопоставь метрику с её значением при визуализации:

🔗 **Подсказка:**

Одна метрика показывает лидера, другая — реплики, третья — отставание.

Ответ:

- **LeaderId** — Идентификатор лидера партии.
- **Replicas** — Все реплики партии.
- **Isr** — Реплики, синхронизированные с лидером.
- **Lag** — Отставание фолловера от лидера.

🔗 Шаг 08:

Вопрос 7

Программист заметил, что lag быстро растёт. Что это значит?

Подсказка: Лаг связан с репликацией сообщений.

Ответ:

- **Фолловеры не успевают догонять лидера**
- Лидер потерял все сообщения
- Кластер остановился
- Произошла смена контроллера

🔔 **Шаг 09:**

Вопрос 8

Расставь шаги работы лидера и фолловеров в правильном порядке:

🔔 **Подсказка:**

Сначала лидер принимает запись, потом фолловеры синхронизируются.

Ответ:

- Продюсер отправляет сообщение лидеру
- Лидер записывает сообщение
- Фолловеры копируют запись
- ISR обновляется

🔔 **Шаг 10:**

Вопрос 9

Сопоставь ситуацию и то, что произойдёт:

🔔 **Подсказка:**

Подумай про выбор лидера и ISR.

Ответ:

- **Лидер жив и здоров** — Фолловеры синхронизируются с ним.
- **Лидер упал, ISR не пуст** — Выбирается новый лидер из ISR.
- **Лидер упал, ISR пуст** — Данные могут потеряться.
- **Фолловер долго отстаёт** — Его могут исключить из ISR.

🔔 **Шаг 11:**

Вопрос 10

Ты визуализировал топик с тремя репликами. Как выглядит здоровая ситуация?

Подсказка: Лидер + синхронные реплики = надёжность.

Ответ:

- **Есть один лидер**
- **Все реплики входят в ISR**
- **Lag минимален**
- Два лидера одновременно

📖 **Урок 3 - Практическая игра «найди потерянное сообщение»**

🔔 **Шаг 02:**

Вопрос 1

Ты видишь, что сообщение не дошло до потребителя. Что первым делом стоит проверить?

Подсказка: Сначала проверяем, дошло ли сообщение до брокера.

Ответ:

- **Проверить, что продюсер успешно отправил сообщение**
- Сразу перезапустить весь кластер
- Удалить все топики
- Переписать код потребителя

🔗 **Шаг 03:**

Вопрос 2

Если сообщение до брокера дошло, но потребитель его не видит, что проверить дальше?

Подсказка: Смотри смещения (offset) потребителя.

Ответ:

- **Проверить, не отстал ли потребитель по offset**
- Удалить сообщения в топике
- Проверить, есть ли лидер у партии
- Изменить ключи сообщений

🔗 **Шаг 04:**

Вопрос 3

Сопоставь элемент Kafka с его ролью:

🔗 **Подсказка:**

Подумай, где хранится сообщение, кто его рассылает и кто догоняет.

Ответ:

- **Продюсер** — Отправляет сообщение в топик.
- **Брокер** — Хранит сообщения и рассылает их потребителям.
- **Потребитель** — Считывает сообщения с определенного offset.
- **Фолловер** — Синхронизирует данные с лидером, чтобы быть готовым к отказу.

🔗 **Шаг 05:**

Вопрос 4

Расположи шаги поиска потерянного сообщения:

🔗 **Подсказка:**

Сначала проверяем путь сообщения от продюсера до потребителя.

Ответ:

- Проверить, дошло ли сообщение до брокера
- Проверить offset потребителя
- Сравнить смещения лидера и фолловеров
- Проверить лог продюсера и потребителя

🔗 **Шаг 06:**

Вопрос 5

Как определить, что сообщение потерялось из-за фолловера?

Подсказка: Смотри lag и ISR.

Ответ:

- **Lag у фолловера сильно растёт, а он не входит в ISR**

- Фолловер стал лидером
- Все сообщения дошли до всех потребителей
- Фолловер выключен навсегда

🔊 Шаг 07:

Вопрос 6

Сопоставь причины потери сообщения и последствия:

🔊 Подсказка:

Подумай, как отставание фолловера влияет на ISR.

Ответ:

- **Фолловер долго отстаёт** — Может быть исключён из ISR
- **Лидер упал** — Выбирается новый лидер из ISR
- **Потребитель пропустил offset** — Не увидит сообщение
- **Сообщение не дошло до брокера** — Будет потеряно

🔊 Шаг 08:

Вопрос 8

Расположи этапы от отправки сообщения до его получения потребителем:

🔊 Подсказка:

Сначала продюсер пишет, затем лидер распределяет, потом фолловеры синхронизируют.

Ответ:

- Продюсер отправляет сообщение
- Лидер сохраняет и рассылает
- Фолловеры копируют сообщение
- Потребитель считывает сообщение

🔊 Шаг 09:

Вопрос 9

Сопоставь визуализацию и её интерпретацию:

🔊 Подсказка:

Lag и статус реплики помогают понять, есть ли потеря сообщений.

Ответ:

- **Lag = 0, ISR включает все реплики** — Все сообщения синхронизированы
- **Lag растёт, реплика вне ISR** — Может потерять сообщения
- **Лидер сменился, ISR неполный** — Некоторые реплики отстали
- **Потребитель видит старый offset** — Пропустил сообщение